HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO ASSIGNMENT 06

HỌC PHẦN: PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Giảng viên	**: PGS.TS. Trần Đình Quế**
Nhóm học phần	**: 05**
Họ và tên	**: Nguyễn Đức Trung**
Mã sinh viên	**: B23DCCN857**
Lớp	**: D23CNPM01**

HÀ NỘI 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU VÀ YÊU CẦU ASSIGNMENT 06 7

1.1. Bối cảnh bài toán 7

1.2. Bốn yêu cầu chính của Assignment 06 7

1.3. Phạm vi và nguyên tắc thực nghiệm 8

1.4. Quy trình tổng thể 8

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 9

2.1. Dữ liệu tuần tự và bài toán dự báo 9

2.2. RNN và ý tưởng bộ nhớ trạng thái 9

2.3. Công thức cập nhật hidden state 10

2.4. Timestep, sequence length và hidden state 10

2.5. Dạng tensor đầu vào 10

2.6. Sliding window 11

2.7. Chuẩn hóa dữ liệu 11

2.8. Hàm mất mát và tối ưu 11

2.9. Các metric đánh giá 12

2.10. Overfitting trong RNN 12

2.11. PyTorch RNN và Keras SimpleRNN 13

CHƯƠNG 3. DỮ LIỆU, MÔI TRƯỜNG VÀ THIẾT KẾ THỰC NGHIỆM 13

3.1. Dataset 1 – Online Retail Business 13

3.2. Khám phá dữ liệu Customer Behavior 15

3.3. Xử lý cancellation và giá không hợp lệ 15

3.4. Tạo chuỗi Customer Behavior theo ngày 16

3.5. Dataset 2 – S&P 500 Stock Data 20

3.6. Khám phá AAPL 20

CHƯƠNG 4. PHƯƠNG PHÁP THỰC NGHIỆM 23

4.1. Chuẩn hóa quy trình cho hai dataset 23

4.2. Chia train – validation – test 24

4.3. Tách scaler cho feature và target 25

4.4. Kiến trúc BasicRNN 25

4.5. Huấn luyện 26

4.6. Code tạo sequence 26

4.7. Code đánh giá 26

CHƯƠNG 5. THỰC NGHIỆM PYTORCH TRÊN CUSTOMER BEHAVIOR 27

5.1. Chuẩn bị dữ liệu 27

5.2. Huấn luyện và đường cong loss 27

5.3. Dự đoán và metric 28

5.4. Nhận xét 30

CHƯƠNG 6. THỰC NGHIỆM PYTORCH TRÊN AAPL 30

6.1. Chuẩn bị dữ liệu 30

6.2. Huấn luyện 30

6.3. Kết quả test 31

6.4. Nhận xét 33

CHƯƠNG 7. THỰC NGHIỆM KERAS TRÊN CUSTOMER BEHAVIOR 33

7.1. Kiến trúc SimpleRNN 33

7.2. Huấn luyện và đánh giá 34

7.3. Kết quả 34

7.4. So sánh nhanh Customer Behavior 35

CHƯƠNG 8. THỰC NGHIỆM KERAS TRÊN AAPL 36

8.1. Cấu hình 36

8.2. Huấn luyện 36

8.3. Kết quả test 37

8.4. Nhận xét về warning TensorFlow 38

CHƯƠNG 9. SO SÁNH PYTORCH VÀ KERAS 38

9.1. Bảng kết quả tổng hợp 38

9.2. So sánh theo Customer Behavior 39

9.3. So sánh theo AAPL 39

9.4. Vì sao không so sánh RMSE giữa hai dataset? 41

9.5. Ý nghĩa của R^2 âm 41

9.6. Phân tích khả năng khái quát hóa 41

9.7. So sánh framework cần được diễn giải có điều kiện 41

9.8. Mã tạo bảng so sánh 42

9.9. Trả lời trực tiếp bốn yêu cầu của Assignment 06 43

CHƯƠNG 10. ĐÁNH GIÁ, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN 44

10.1. Điểm mạnh của thiết kế thực nghiệm 44

10.2. Hạn chế của Customer Behavior 44

10.3. Hạn chế của AAPL 44

10.4. Hạn chế về reproducibility 45

10.5. Hướng phát triển 45

10.6. Sơ đồ phát triển đề xuất cho phiên bản nâng cao 45

CHƯƠNG 11. KẾT LUẬN 46

TÀI LIỆU THAM KHẢO **47**

Liên kết nguồn **47**

PHỤ LỤC A. SOURCE CODE VÀ GIẢI THÍCH CÁC KHỐI CODE **49**

Cell 7 – Source code **49**

Cell 13 – Source code **49**

Cell 16 – Source code **50**

Cell 24 – Source code **51**

Cell 26 – Source code **51**

Cell 31 – Source code **51**

Cell 33 – Source code **51**

Cell 37 – Source code **52**

Cell 51 – Source code **52**

Cell 52 – Source code **53**

Cell 53 – Source code **54**

Cell 54 – Source code **54**

Cell 60 – Source code **55**

Cell 64 – Source code **55**

Cell 66 – Source code **56**

Cell 68 – Source code **57**

Cell 69 – Source code **57**

Cell 72 – Source code **58**

Cell 78 – Source code **58**

Cell 80 – Source code **58**

Cell 88 – Source code **59**

Cell 89 – Source code **59**

Cell 90 – Source code **60**

Cell 91 – Source code **61**

Cell 92 – Source code **62**

Cell 97 – Source code **62**

Cell 99 – Source code **63**

Cell 101 – Source code **64**

Cell 103 – Source code **64**

Cell 107 – Source code **65**

Cell 109 – Source code **65**

Cell 110 – Source code **66**

Cell 112 – Source code **67**

Cell 113 – Source code **67**

Cell 117 – Source code **68**

Cell 118 – Source code **69**

Cell 120 – Source code **69**

Cell 121 – Source code **70**

Cell 125 – Source code **70**

Cell 131 – Source code **71**

PHỤ LỤC B. THÔNG TIN DATASET VÀ LIÊN KẾT **73**

CHƯƠNG 1. GIỚI THIỆU VÀ YÊU CẦU ASSIGNMENT 06

1.1. Bối cảnh bài toán

Dữ liệu theo thời gian xuất hiện trong nhiều hệ thống thực tế như giao dịch bán hàng, lưu lượng truy cập, cảm biến IoT, nhu cầu khách hàng, giá cổ phiếu và văn bản. Điểm khác biệt của dữ liệu chuỗi thời gian so với dữ liệu bảng thông thường là thứ tự của các quan sát có ý nghĩa. Một giá trị ở thời điểm hiện tại có thể phụ thuộc vào những giá trị ở các thời điểm trước đó. Vì vậy, một mô hình chỉ xem từng dòng độc lập có thể bỏ qua thông tin quan trọng nằm trong quan hệ tuần tự.

Assignment 06 yêu cầu nghiên cứu mạng RNN ở mức cơ bản và triển khai trên hai dạng dữ liệu tuần tự khác nhau. Notebook đã chọn một dataset giao dịch bán lẻ và một dataset chứng khoán. Hai lựa chọn này tạo ra hai bối cảnh khác nhau: dữ liệu bán lẻ được biến đổi từ giao dịch cấp dòng thành hành vi kinh doanh theo ngày, còn dữ liệu chứng khoán vốn đã có cấu trúc theo phiên giao dịch.

Mục tiêu không phải xây dựng một hệ thống dự báo tài chính hay thương mại hoàn chỉnh, mà là minh họa đầy đủ vòng đời của một bài toán RNN: hiểu dữ liệu, chuẩn hóa, tạo chuỗi, đưa vào mô hình, huấn luyện, đánh giá và trực quan hóa. Vì vậy, báo cáo ưu tiên tính nhất quán giữa các bước và khả năng giải thích code hơn việc tìm một cấu hình tối ưu.

1.2. Bốn yêu cầu chính của Assignment 06

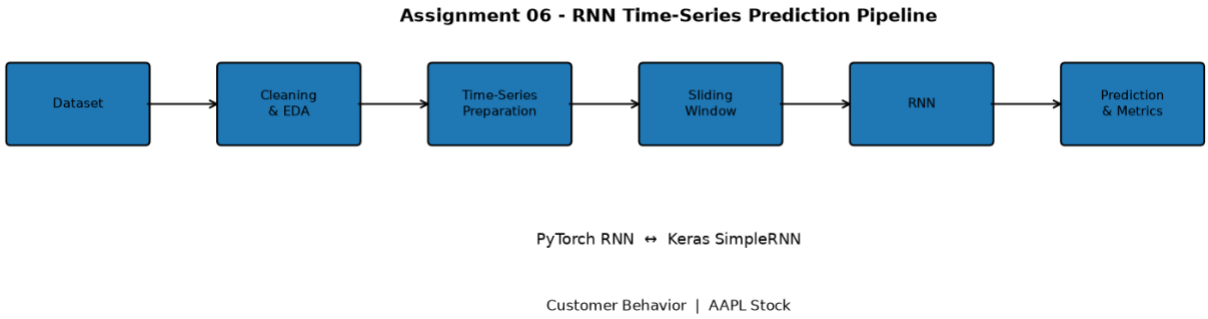
Yêu cầu	Cách đáp ứng trong notebook
1. Cơ sở RNN, hàm và code	Trình bày RNN, sequential data, timestep, hidden state, công thức cập nhật trạng thái, input shape và mô hình BasicRNN bằng PyTorch.
2. Hai dataset chuỗi thời gian	Online Retail Business được biến đổi thành Customer Behavior theo ngày; S&P

	500 được lọc thành chuỗi AAPL.
3. PyTorch: phân tích – dự đoán – trực quan hóa	EDA, chuẩn bị sequence, BasicRNN, train/validation, dự đoán test, MAE/MSE/RMSE/R^2 và biểu đồ.
4. Keras: phân tích – dự đoán – trực quan hóa	SimpleRNN + Dense trên cùng sequence; huấn luyện, dự đoán, đánh giá và trực quan hóa cho cả hai dataset.

1.3. Phạm vi và nguyên tắc thực nghiệm

- Chỉ sử dụng basic RNN/SimpleRNN, không thay thế bằng LSTM hoặc GRU trong phần thực nghiệm chính.
- Giữ sequence length = 20 cho cả hai dataset.
- Chia dữ liệu theo thời gian, không shuffle dữ liệu trước khi chia train/validation/test.
- Fit scaler trên train và chỉ transform validation/test.
- Dùng cùng bộ metric MAE, MSE, RMSE và R^2 để so sánh.
- Không điều chỉnh kết quả sau khi chạy để tạo ra chỉ số đẹp hơn.

1.4. Quy trình tổng thể



Hình 1. Pipeline tổng thể của Assignment 06

Nhận xét: Sơ đồ thể hiện chuỗi: Dataset → Cleaning/EDA → Time-series preparation → Chronological split → Scaling → Sliding window → PyTorch RNN/Keras SimpleRNN → Prediction → Metrics → Visualization →

Comparison. Hình này giúp người đọc nhìn thấy toàn bộ quy trình trước khi đi vào từng chương.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Dữ liệu tuần tự và bài toán dự báo

Dữ liệu tuần tự là dữ liệu mà thứ tự quan sát mang thông tin. Với một chuỗi x_1, x_2,…, x_t, mô hình không chỉ quan tâm từng x_i riêng lẻ mà còn quan tâm mối liên hệ giữa xi và các phần tử trước đó. Trong bài toán dự báo, một cửa sổ gồm nhiều bước quá khứ được dùng làm đầu vào để dự đoán giá trị tương lai. Cách biểu diễn này phù hợp với cả doanh thu theo ngày và giá đóng cửa theo phiên giao dịch.

Trong notebook, một timestep của Customer Behavior tương ứng với một ngày có dữ liệu giao dịch. Một timestep của AAPL tương ứng với một phiên giao dịch. Do dữ liệu giao dịch bán lẻ không có giao dịch ở mọi ngày lịch, chuỗi Customer Behavior thực chất là chuỗi của các ngày xuất hiện trong dữ liệu sau tổng hợp, thay vì một chuỗi lịch liên tục có chèn số 0 cho những ngày không có giao dịch. Cách xử lý này giữ lại các quan sát thực tế và tránh tự ý giả định doanh thu bằng 0.

2.2. RNN và ý tưởng bộ nhớ trạng thái

RNN là một lớp mạng nơ-ron được thiết kế cho dữ liệu tuần tự. Ý tưởng trung tâm là tại mỗi timestep, mạng nhận đầu vào hiện tại cùng hidden state của timestep trước để tạo ra hidden state mới. Cơ chế này cho phép thông tin từ quá khứ được truyền qua chuỗi. Elman trình bày một dạng mạng hồi quy trong đó trạng thái bên trong được tái sử dụng để biểu diễn thông tin phụ thuộc ngữ cảnh theo thời gian [1].

Trong phạm vi Assignment 06, RNN được sử dụng ở dạng đơn giản: một lớp RNN xử lý toàn bộ sequence, sau đó output tại timestep cuối được đưa qua lớp tuyến tính để tạo một giá trị dự đoán. Cách thiết kế này phù hợp với bài toán many-to-one regression, trong đó một chuỗi quá khứ dẫn đến một giá trị mục tiêu duy nhất.

2.3. Công thức cập nhật hidden state

Công thức được trình bày trong notebook là:

$$\mathbf{h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)}$$

Trong đó x_t là vector đặc trưng tại timestep hiện tại, h_{t-1} là hidden state của timestep trước, W_{xh} là ma trận trọng số từ input đến hidden, W_{hh} là ma trận hồi quy từ hidden trước đó đến hidden hiện tại và b_h là bias. Hàm tanh đưa giá trị về khoảng giới hạn và tạo phi tuyến. Tài liệu PyTorch mô tả nn.RNN như một Elman RNN có thể dùng tanh hoặc ReLU; cấu hình notebook giữ mặc định tanh [2].

2.4. Timestep, sequence length và hidden state

Sequence length là số timestep được đưa vào mỗi mẫu huấn luyện. Notebook chọn sequence length = 20. Như vậy, một mẫu Customer Behavior gồm 20 ngày liên tiếp trong chuỗi đã tổng hợp; một mẫu AAPL gồm 20 phiên giao dịch. Mục tiêu nằm ngay sau cửa sổ này, tức ngày hoặc phiên kế tiếp.

Hidden size = 64 nghĩa là tại mỗi timestep, RNN duy trì một vector trạng thái có 64 phần tử. Hidden size không phải số feature đầu vào. Với Customer Behavior và AAPL, input size đều bằng 5; hidden size bằng 64; output cuối cùng là một số thực thông qua Linear/Dense.

2.5. Dạng tensor đầu vào

Notebook sử dụng batch_first=True trong PyTorch nên input có dạng (batch_size, sequence_length, input_size). Đây cũng là cách Keras SimpleRNN tiếp nhận tensor ba chiều có dạng (batch, timesteps, feature) [2], [3]. Với thí nghiệm AAPL, shape của tập train sau khi tạo sequence là (861, 20, 5). Với Customer Behavior, shape tương ứng là (193, 20, 5).

Thành phần	Ý nghĩa	Giá trị trong thí nghiệm

batch_size	Số sequence xử lý trong một batch	16
sequence_length	Số timestep trong một sequence	20
input_size	Số đặc trưng tại mỗi timestep	5
hidden_size	Kích thước hidden state	64
output_size	Số giá trị dự đoán	1

2.6. Sliding window

Để biến chuỗi thành dữ liệu học có giám sát, notebook sử dụng hàm create_sequences. Với sequence length L, tại vị trí i mô hình lấy X[i:i+L] làm input và y[i+L] làm target. Vì vậy, từ N quan sát có thể tạo N-L sequence. Đây là lý do Customer Behavior từ 213 mẫu train còn 193 sequence, còn AAPL từ 881 mẫu train còn 861 sequence.

2.7. Chuẩn hóa dữ liệu

Các feature có đơn vị và độ lớn rất khác nhau. Trong AAPL, volume có thể ở mức hàng chục hoặc hàng trăm triệu, trong khi giá chỉ ở mức vài chục đến vài trăm USD. StandardScaler chuẩn hóa từng feature theo z = (x - u)/s, trong đó u và s được tính từ tập train [4]. Notebook dùng một scaler riêng cho feature và một scaler riêng cho target. Quan trọng hơn, scaler chỉ được fit trên train rồi dùng transform cho validation/test.

2.8. Hàm mất mát và tối ưu

Bài toán là hồi quy nên notebook dùng Mean Squared Error (MSE) làm loss. MSE bình phương sai số giữa giá trị thật và dự đoán, do đó các sai số lớn có ảnh hưởng mạnh

hơn [6]. Optimizer Adam với learning rate 0,001 được dùng cho cả PyTorch và Keras. Việc dùng cùng learning rate giúp giảm một nguồn khác biệt khi so sánh framework.

2.9. Các metric đánh giá

Metric	Ý nghĩa	Cách diễn giải
MAE	Trung bình trị tuyệt đối của sai số	Càng nhỏ càng tốt; cùng đơn vị với target.
MSE	Trung bình bình phương sai số	Càng nhỏ càng tốt; nhạy với sai số lớn.
RMSE	Căn bậc hai của MSE	Cùng đơn vị với target, dễ diễn giải hơn MSE.
R^2	Hệ số xác định	Tối đa 1; có thể âm nếu mô hình kém hơn baseline trung bình.

Theo tài liệu scikit-learn, MAE là một loss hồi quy không âm và giá trị tốt nhất là 0 [5]. MSE cũng không âm và giá trị tốt nhất là 0 [6]. R^2 có giá trị tốt nhất bằng 1 nhưng có thể âm; một mô hình luôn dự đoán giá trị trung bình của y có R^2 bằng 0 trong trường hợp thông thường [7]. Vì vậy, các giá trị R^2 âm trong notebook không phải lỗi tính toán tự động; chúng phản ánh chất lượng dự báo trên test theo đúng định nghĩa metric.

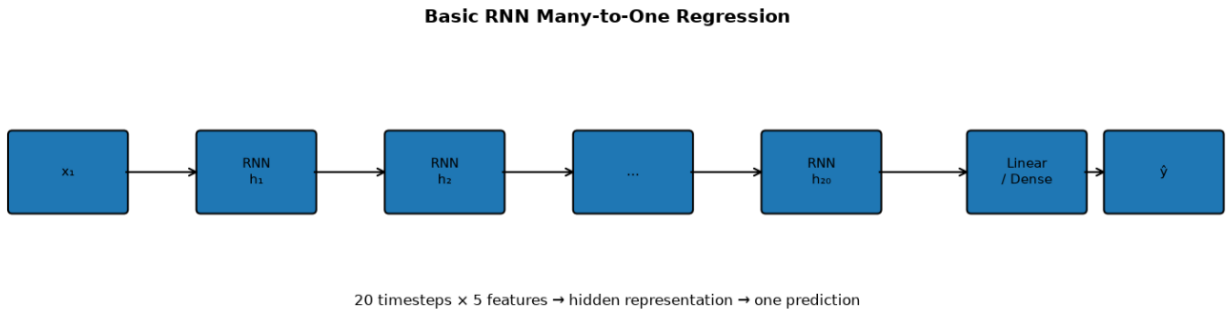
2.10. Overfitting trong RNN

Overfitting xảy ra khi loss trên train tiếp tục giảm nhưng loss trên validation không giảm tương ứng hoặc bắt đầu tăng. Đây là hiện tượng có thể quan sát rõ trong một số đường cong của notebook. Đặc biệt, với Customer Behavior PyTorch, train loss giảm liên tục trong khi validation loss vẫn quanh mức cao và dao động. Với AAPL Keras, validation loss đạt mức thấp sớm rồi tăng dần trong khi training loss tiếp tục giảm. Những

dấu hiệu này cho thấy số epoch cố định 50 không nhất thiết là thời điểm tối ưu cho mọi dataset.

2.11. PyTorch RNN và Keras SimpleRNN

PyTorch sử dụng torch.nn.RNN. Theo tài liệu PyTorch, lớp này thực hiện RNN nhiều lớp theo dạng Elman và hỗ trợ tanh hoặc ReLU [2]. Notebook xây dựng BasicRNN bằng một nn.RNN và một nn.Linear. Keras sử dụng SimpleRNN, một lớp RNN fully-connected trong đó output được đưa hồi tiếp vào mạng; lớp này nhận tensor 3 chiều (batch, timesteps, feature) [3]. Vì hai mô hình được xây dựng cùng hidden size, sequence length, input features và Dense/Linear output, việc so sánh là hợp lý ở mức bài tập cơ bản.



Hình 2. Cấu trúc Basic RNN many-to-one

Nhận xét: Sơ đồ thể hiện 20 timestep đi qua RNN, hidden state được truyền từ t-1 sang t, output cuối cùng đi vào Linear/Dense để tạo một giá trị dự đoán.

CHƯƠNG 3. DỮ LIỆU, MÔI TRƯỜNG VÀ THIẾT KẾ THỰC NGHIỆM

3.1. Dataset 1 – Online Retail Business

Notebook sử dụng file OnlineRetail.csv thuộc dataset Online Retail Business trên Kaggle. Dataset được mô tả là dữ liệu các đơn hàng trực tuyến với các trường InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID và Country [8]. Trong notebook thực tế, file có 541.909 dòng và 8 cột. UCI Online Retail cũng ghi nhận

một bộ dữ liệu giao dịch cùng cấu trúc và quy mô 541.909 instances, thuộc nhóm dữ liệu sequential/time-series [9].

Mục tiêu của bài tập không phải dự đoán từng giao dịch riêng lẻ mà là xây dựng một chuỗi đại diện cho hoạt động kinh doanh theo ngày. Vì vậy, dữ liệu cấp transaction được chuyển thành năm biến tổng hợp: doanh thu trong ngày, số invoice khác nhau, số khách hàng có CustomerID, tổng quantity và giá đơn vị trung bình.

Thuộc tính	**Vai trò trong notebook**	**Ý nghĩa**
InvoiceNo	Xác định giao dịch/hóa đơn	Dùng để đếm Transactions và phát hiện cancellation.
StockCode	Mã sản phẩm	Không dùng trực tiếp trong RNN cuối.
Description	Tên sản phẩm	Phục vụ hiểu dữ liệu, không dùng làm feature.
Quantity	Số lượng	Được tổng hợp thành Quantity theo ngày.
InvoiceDate	Thời gian	Dùng để tạo Date và chuỗi thời gian.
UnitPrice	Đơn giá	Dùng tính Revenue và AvgUnitPrice.
CustomerID	Mã khách hàng	Dùng đếm khách hàng duy nhất.
Country	Quốc gia	Không dùng trực tiếp trong RNN cuối.

3.2. Khám phá dữ liệu Customer Behavior

Kết quả notebook cho thấy dữ liệu ban đầu có 541.909 dòng. CustomerID thiếu 135.080 dòng, Description thiếu 1.454 dòng. Có 5.268 dòng trùng lặp. Quantity có giá trị âm và UnitPrice có cả giá trị âm hoặc bằng 0. Những đặc điểm này cho thấy dữ liệu giao dịch thực tế chứa các bản ghi cần xử lý trước khi xây dựng time series.

Chỉ tiêu	Kết quả
Số dòng ban đầu	541.909
Số cột	8
Duplicate rows	5.268
Missing CustomerID	135.080
Thời điểm bắt đầu	2010-12-01 08:26:00
Thời điểm kết thúc	2011-12-09 12:50:00
Số dòng invoice cancellation	9.288
Số dòng UnitPrice không hợp lệ sau bước cancellation	2.517
Số dòng sau làm sạch chính	530.104
Missing CustomerID sau làm sạch	132.220

3.3. Xử lý cancellation và giá không hợp lệ

Notebook nhận diện invoice cancellation bằng việc kiểm tra InvoiceNo bắt đầu bằng ký tự C. Có 9.288 dòng cancellation. Các dòng này được loại bỏ để chuỗi Revenue đại diện cho doanh thu bán hàng hợp lệ thay vì trộn cả các giao dịch hoàn/hủy.

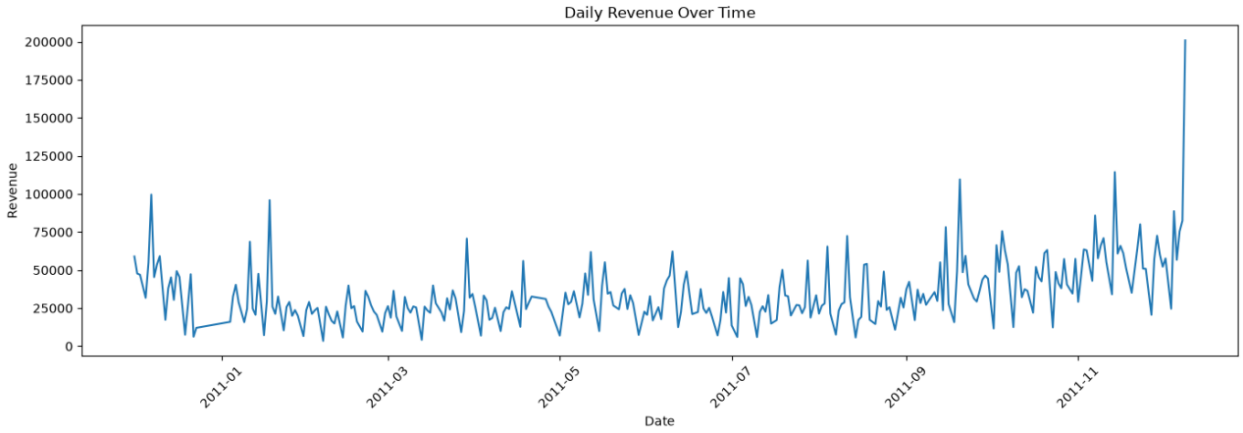
Sau khi loại cancellation, notebook tiếp tục loại các dòng UnitPrice không dương. Có 2.517 dòng bị loại ở bước này. Kết quả còn 530.104 dòng giao dịch sạch. Quantity âm

không còn sau hai bước xử lý. Đây là một ví dụ quan trọng cho yêu cầu phân tích dữ liệu: trước khi áp dụng RNN, cần hiểu ý nghĩa nghiệp vụ của các giá trị bất thường.

3.4. Tạo chuỗi Customer Behavior theo ngày

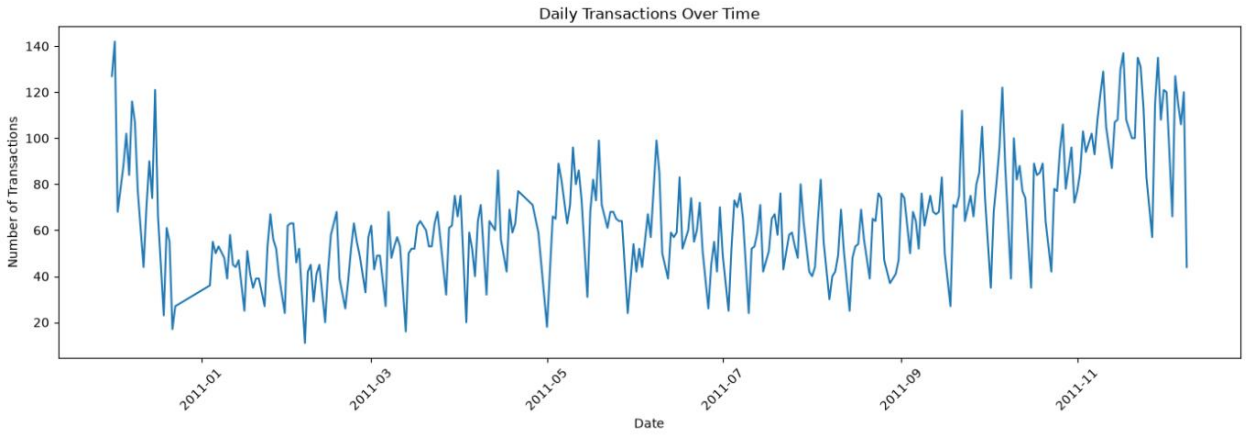
Revenue được tính bằng Quantity × UnitPrice. Sau đó dữ liệu được groupby theo Date và tính Revenue=sum, Transactions=nunique(InvoiceNo), Customers=nunique(CustomerID), Quantity=sum và AvgUnitPrice=mean(UnitPrice). Kết quả có 305 mốc thời gian từ 01/12/2010 đến 09/12/2011.

Feature	Cách tính	Vai trò
Revenue	sum(Quantity × UnitPrice)	Target và đồng thời feature lịch sử.
Transactions	nunique(InvoiceNo)	Mức độ hoạt động giao dịch.
Customers	nunique(CustomerID)	Quy mô khách hàng quan sát được.
Quantity	sum(Quantity)	Khối lượng hàng bán.
AvgUnitPrice	mean(UnitPrice)	Mức giá trung bình.



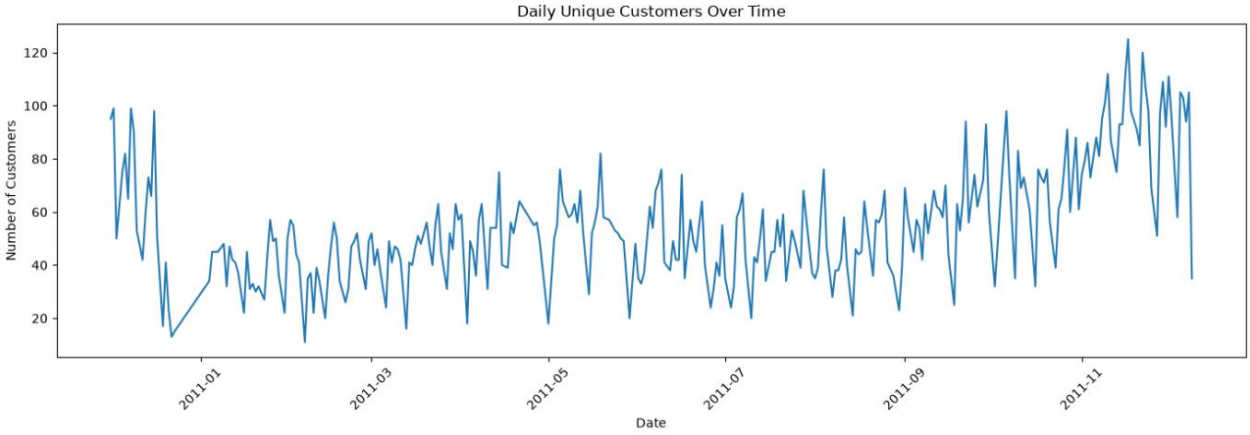
Hình 3. Doanh thu theo ngày của Customer Behavior

Nhận xét: Biểu đồ cho thấy doanh thu theo ngày có sự biến động khá lớn trong năm 2011. Phần lớn doanh thu tập trung ở mức thấp đến trung bình, tuy nhiên xuất hiện một số thời điểm tăng đột biến, đặc biệt vào cuối năm. Nhìn chung, doanh thu có xu hướng tăng nhẹ trong những tháng cuối năm và đạt mức cao nhất khoảng 200.000 vào cuối kỳ. Điều này cho thấy doanh thu không ổn định theo ngày và có thể chịu ảnh hưởng của các thời điểm bán hàng hoặc sự kiện đặc biệt..



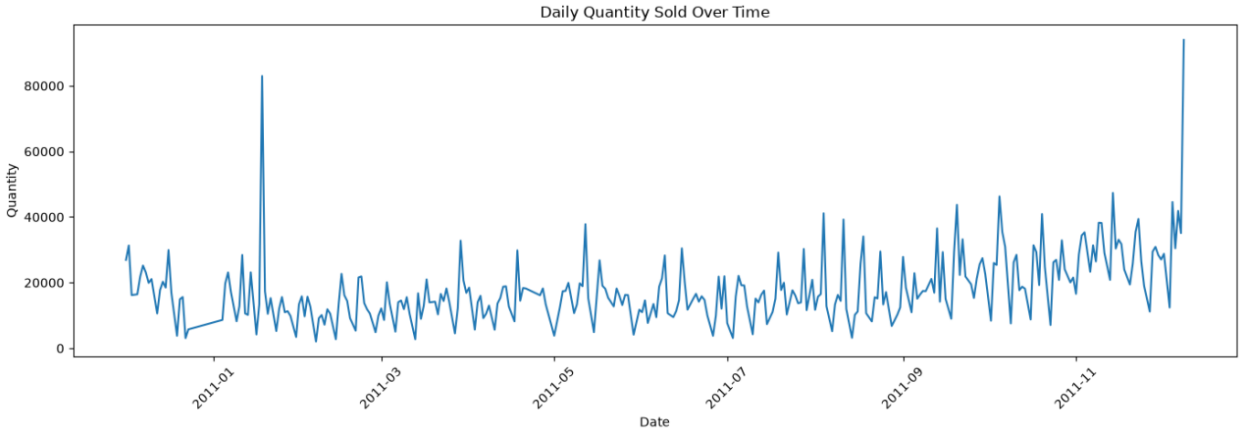
Hình 4. Số giao dịch theo ngày

Nhận xét: Biểu đồ cho thấy số lượng giao dịch theo ngày có sự biến động khá lớn trong năm 2011. Trong giai đoạn đầu năm, số giao dịch dao động mạnh và có một số thời điểm đạt mức cao. Từ giữa năm, số giao dịch chủ yếu dao động trong khoảng 40–80 giao dịch/ngày. Đặc biệt, từ khoảng tháng 9 trở đi, số giao dịch có xu hướng tăng và đạt nhiều mức cao hơn vào cuối năm, cho thấy hoạt động giao dịch có sự gia tăng trong những tháng cuối năm.



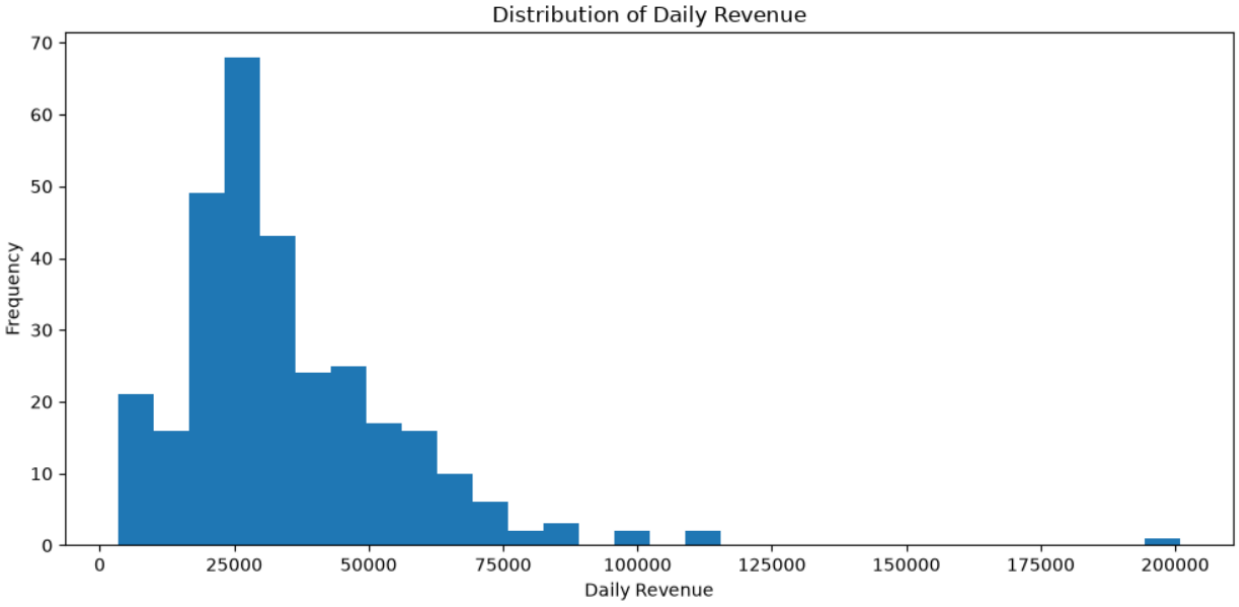
Hình 5. Số khách hàng duy nhất theo ngày

Nhận xét: Biểu đồ thể hiện số lượng khách hàng duy nhất theo ngày và cho thấy sự biến động tương đối lớn trong năm 2011. Phần lớn thời gian trong năm, số khách hàng duy nhất dao động khoảng 20–60 người/ngày. Từ khoảng tháng 9 trở đi, số lượng khách hàng có xu hướng tăng rõ rệt và đạt mức cao nhất vào tháng 11–12, với một số ngày vượt 100 khách hàng. Xu hướng này cho thấy lượng khách hàng hoạt động có sự gia tăng vào cuối năm.



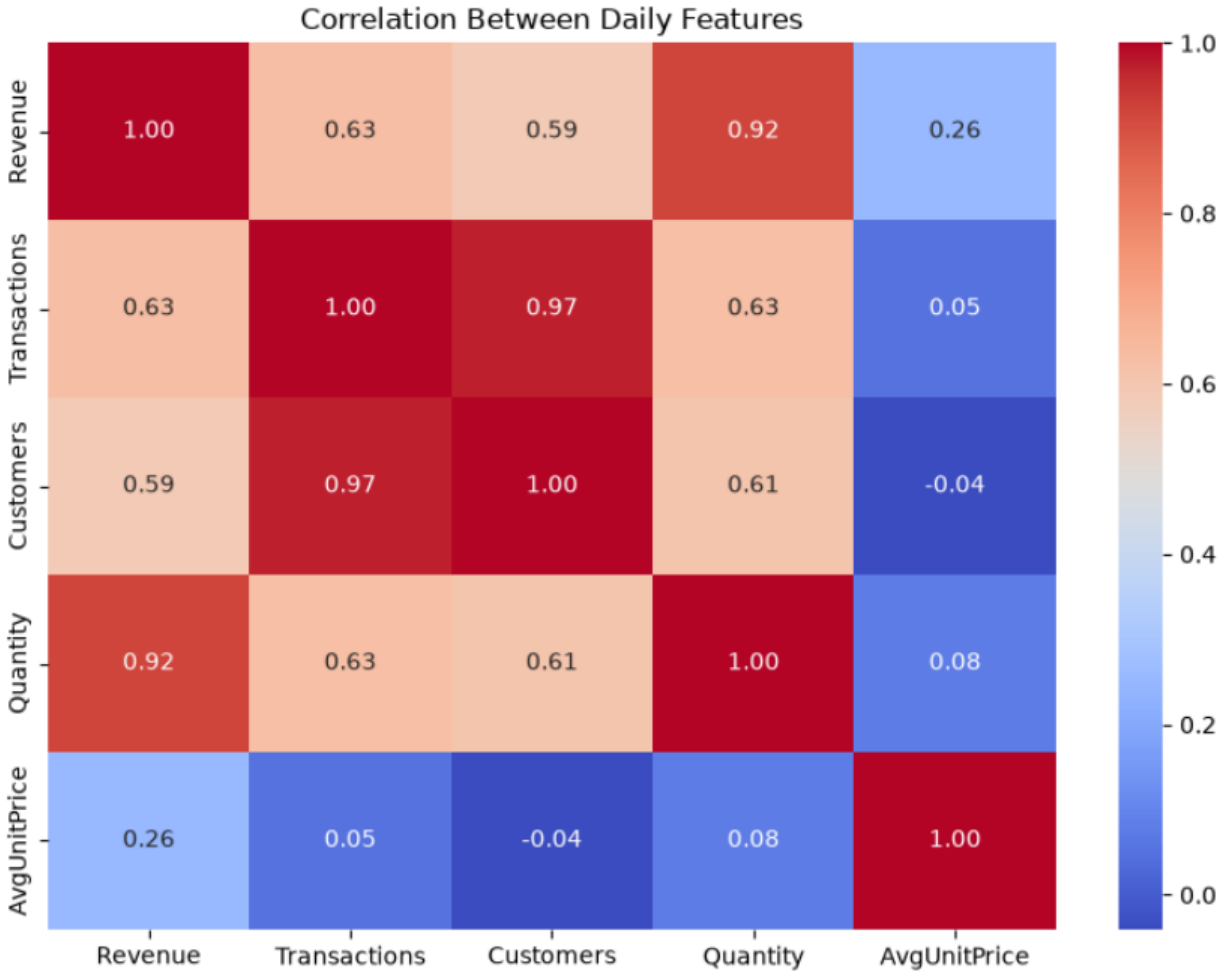
Hình 6. Tổng số lượng hàng bán theo ngày

Nhận xét: Biểu đồ cho thấy số lượng sản phẩm bán ra theo ngày có sự biến động khá lớn trong năm 2011. Phần lớn thời gian, số lượng bán ra dao động ở mức thấp đến trung bình, nhưng xuất hiện một số thời điểm tăng đột biến. Đáng chú ý, từ khoảng tháng 8 trở đi, lượng sản phẩm bán ra có xu hướng tăng và biến động mạnh hơn, đồng thời đạt mức cao nhất vào cuối năm.



Hình 7. Phân phối doanh thu hằng ngày

Nhận xét: Biểu đồ cho thấy doanh thu theo ngày tập trung chủ yếu trong khoảng 15.000–50.000, trong đó nhóm khoảng 20.000–30.000 có tần suất xuất hiện cao nhất. Phân phối có xu hướng lệch phải do xuất hiện một số ngày có doanh thu rất cao, đặc biệt có giá trị ngoại lệ lên tới khoảng 200.000. Điều này cho thấy doanh thu hàng ngày không phân bố đồng đều và một số thời điểm có doanh thu tăng đột biến.



Hình 8. Ma trận tương quan các feature Customer Behavior

Nhận xét: Ma trận tương quan cho thấy Revenue có tương quan dương mạnh với Quantity (0.92), cho thấy doanh thu có mối liên hệ chặt chẽ với số lượng sản phẩm bán ra. Transactions và Customers cũng có tương quan rất mạnh với nhau (0.97), đồng thời đều có tương quan dương với Revenue và Quantity. Ngược lại, AvgUnitPrice có tương quan thấp với Transactions (0.05), Customers (-0.04) và

Quantity (0.08), cho thấy giá bán trung bình có mối liên hệ tuyến tính yếu với các chỉ số hoạt động hàng ngày trong tập dữ liệu.

3.5. Dataset 2 – S&P 500 Stock Data

Dataset thứ hai là S&P 500 stock data trên Kaggle, chứa dữ liệu lịch sử của các công ty trong S&P 500. Trang dataset mô tả file all_stocks_5yr.csv là phiên bản hợp nhất của dữ liệu 5 năm và có các trường date, open, high, low, close, volume và Name [10]. Notebook đọc file này với 619.040 dòng và 7 cột.

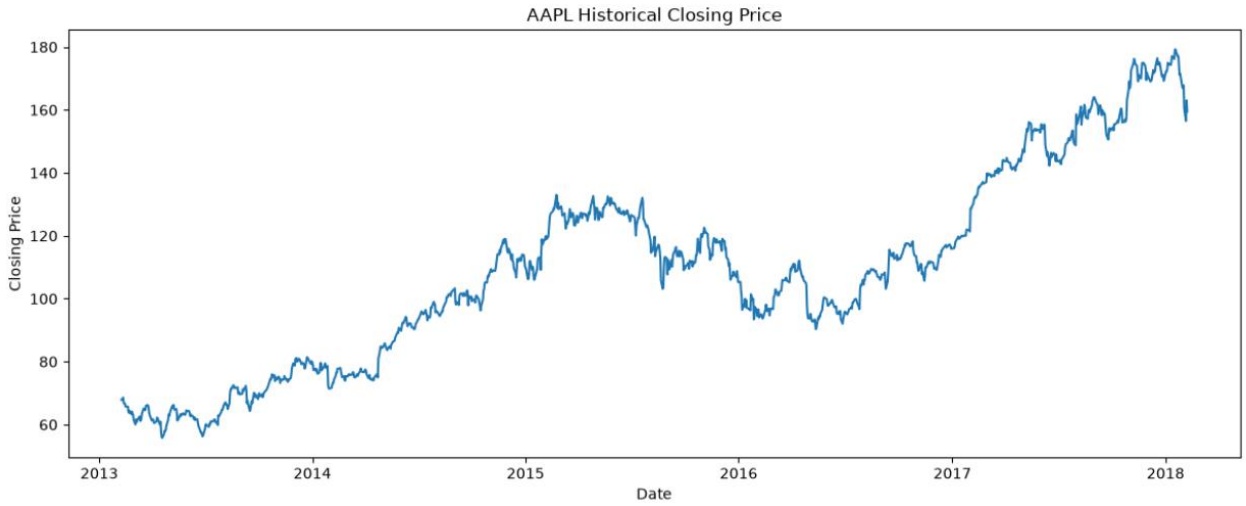
Do file chứa nhiều mã cổ phiếu, notebook chọn AAPL để tạo một bài toán đơn chuỗi. Sau khi lọc Name = AAPL, có 1.259 phiên giao dịch. Khoảng thời gian là 08/02/2013 đến 07/02/2018. Việc chọn một mã duy nhất giúp pipeline RNN rõ ràng, giảm độ phức tạp và phù hợp với mục tiêu minh họa.

Thuộc tính	Vai trò
date	Ngày giao dịch và thứ tự timestep.
open	Giá mở cửa.
high	Giá cao nhất trong phiên.
low	Giá thấp nhất trong phiên.
close	Giá đóng cửa; cũng là target.
volume	Khối lượng cổ phiếu giao dịch.
Name	Mã cổ phiếu.

3.6. Khám phá AAPL

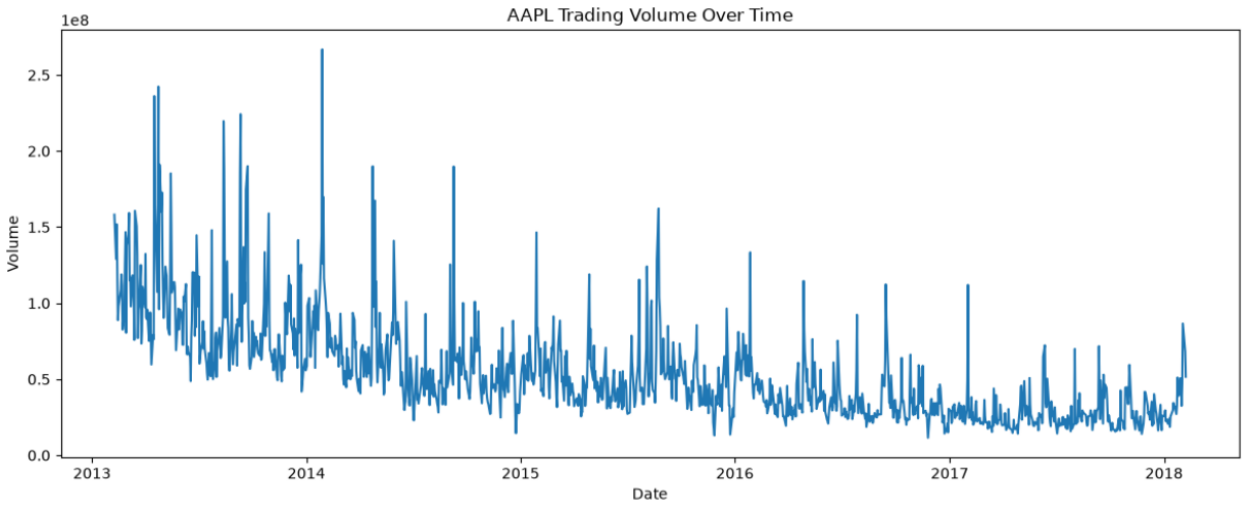
Toàn bộ dataset có 11 giá trị thiếu ở open, 8 ở high và 8 ở low, nhưng sau khi lọc AAPL, 1.259 dòng được dùng trong phần thống kê và modeling không có thiếu ở các trường số quan trọng. Dataset không có duplicate rows. Thống kê AAPL cho thấy close

có trung bình 109,0667, trung vị 109,01, nhỏ nhất 55,7899 và lớn nhất 179,26. Volume có độ phân tán lớn, với trung bình khoảng 54,05 triệu và cực đại khoảng 266,83 triệu.



Hình 9. Giá đóng cửa AAPL theo thời gian

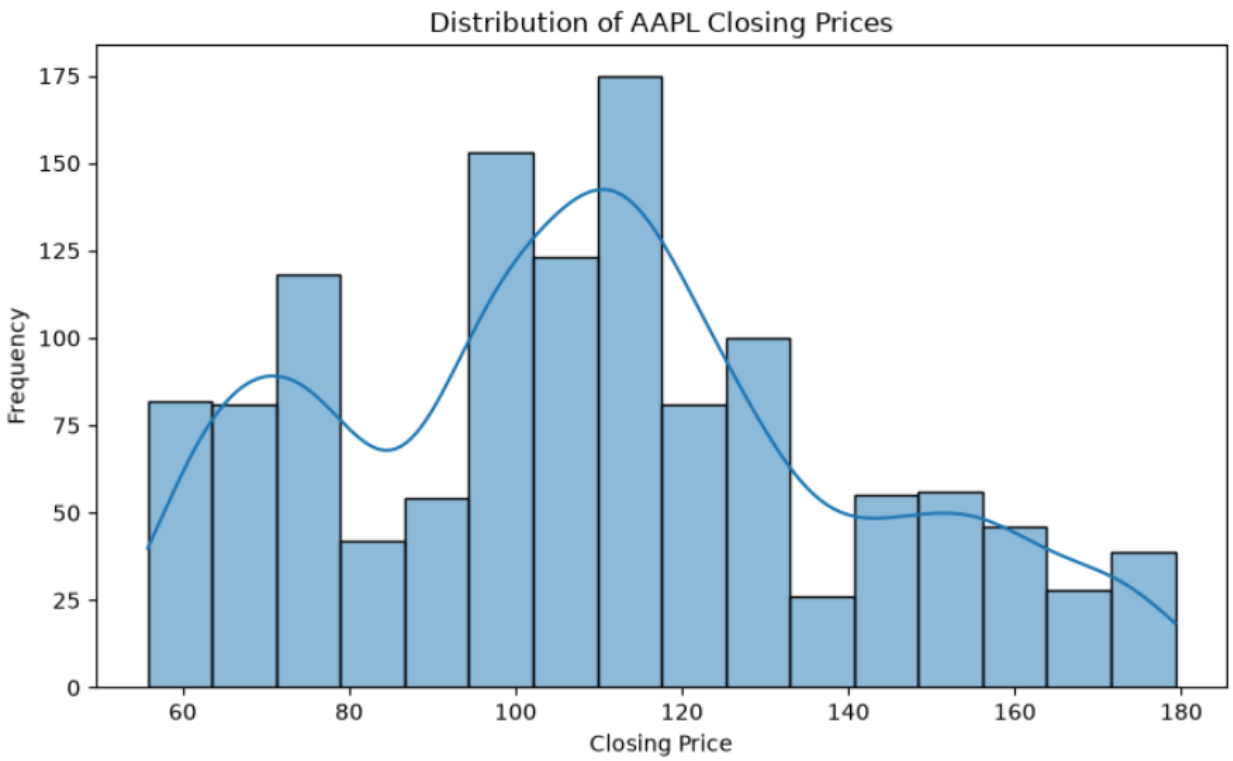
Nhận xét: Biểu đồ cho thấy giá đóng cửa cổ phiếu AAPL có xu hướng tăng trong giai đoạn 2013–2018, mặc dù có nhiều thời điểm biến động và điều chỉnh giảm. Giá tăng từ khoảng 60 vào đầu năm 2013 lên mức cao nhất gần 180 vào đầu năm 2018. Đặc biệt, giai đoạn 2016–2018 ghi nhận xu hướng tăng rõ rệt, cho thấy sự thay đổi đáng kể của giá cổ phiếu theo thời gian.



Hình 10. Khối lượng giao dịch AAPL

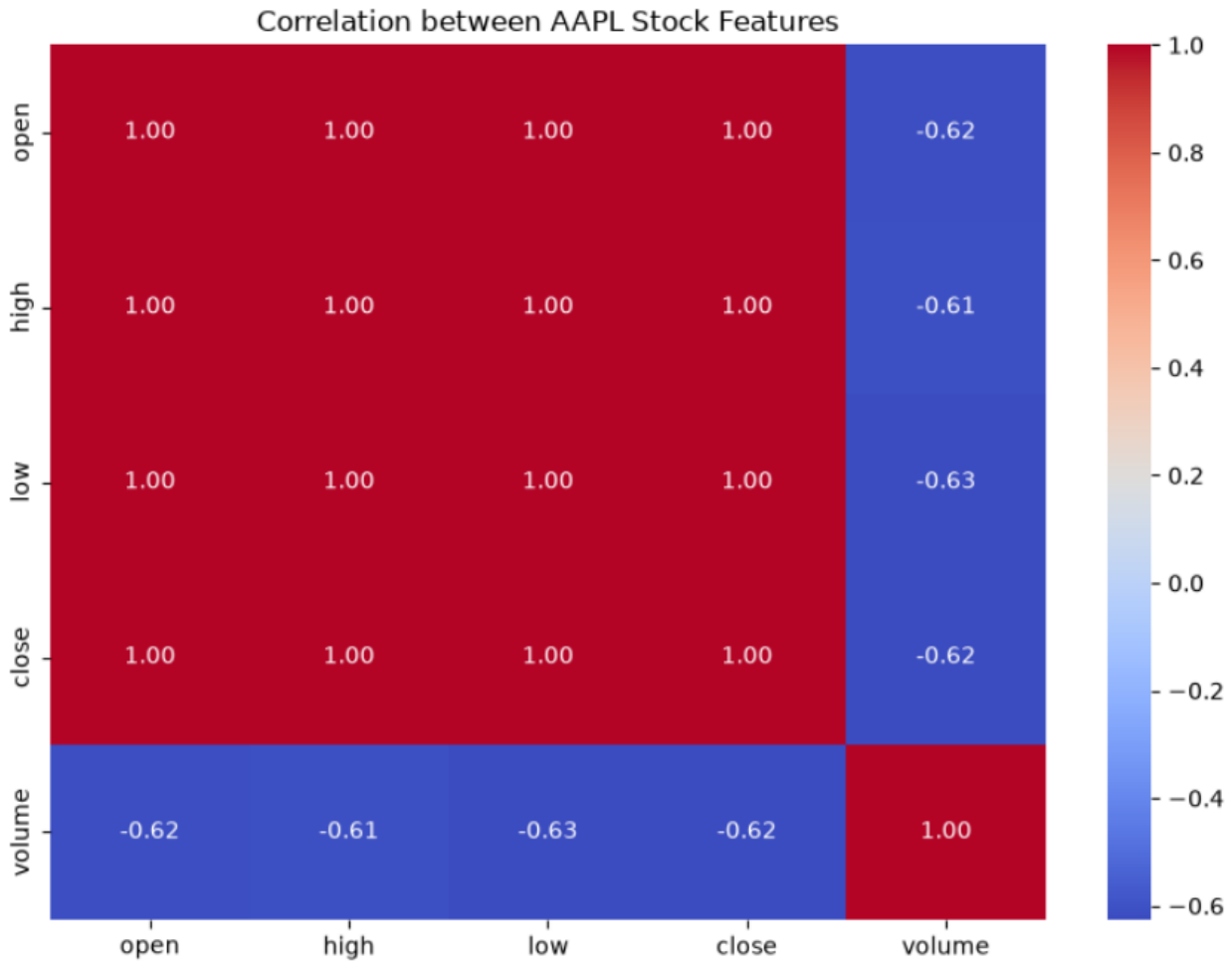
Nhận xét: Khối lượng giao dịch có xu hướng giảm dần trong giai đoạn 2013–2018, mặc dù vẫn xuất hiện nhiều thời điểm tăng đột biến. Giai đoạn đầu có nhiều phiên đạt khối lượng giao dịch rất cao, trong khi những năm sau mức giao dịch

thông thường thấp hơn. Điều này cho thấy khối lượng giao dịch biến động mạnh và không duy trì ổn định theo thời gian.



Hình 11. Phân phối giá đóng cửa AAPL

Nhận xét: Giá đóng cửa AAPL phân bố chủ yếu trong khoảng 90–130, trong đó nhóm giá khoảng 100–120 xuất hiện với tần suất cao. Dữ liệu vẫn trải rộng từ khoảng 55 đến 180, cho thấy giá cổ phiếu có phạm vi biến động tương đối lớn trong toàn bộ giai đoạn quan sát. Phân phối không hoàn toàn đối xứng và có xu hướng trải dài về phía các mức giá cao.



Hình 12. Tương quan giữa các biến AAPL

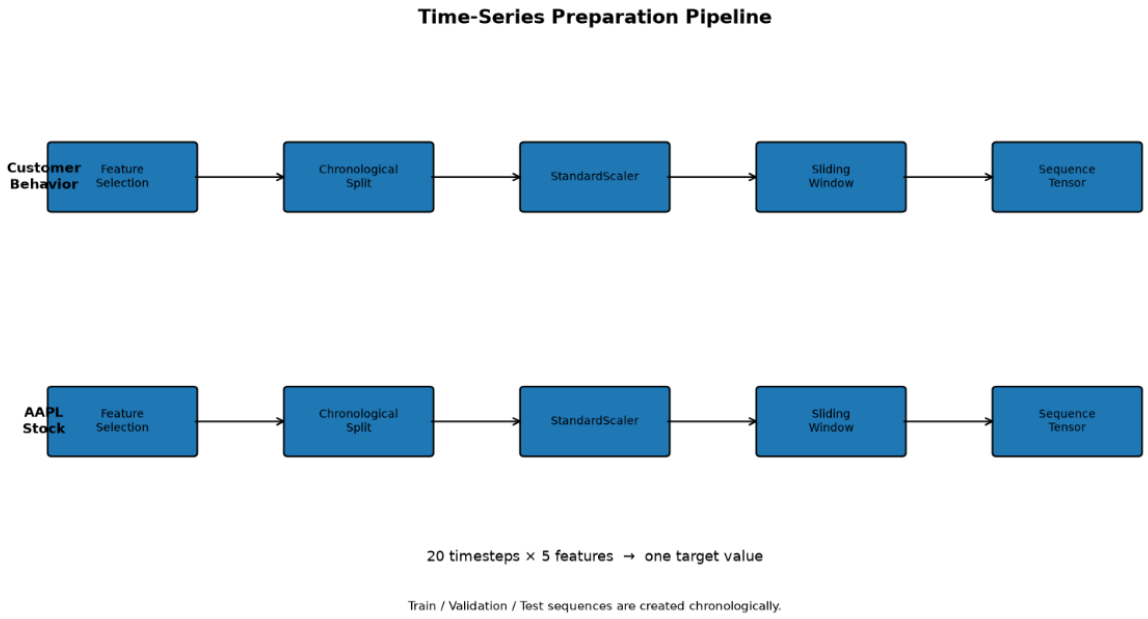
Nhận xét: Ma trận tương quan cho thấy các biến Open, High, Low và Close có tương quan dương gần như tuyệt đối với nhau, với hệ số tương quan xấp xỉ 1. Điều này phản ánh mối quan hệ rất chặt chẽ giữa các mức giá trong cùng một phiên giao dịch. Trong khi đó, Volume có tương quan âm ở mức trung bình với các biến giá, dao động khoảng -0,61 đến -0,63, cho thấy trong tập dữ liệu này khối lượng giao dịch có xu hướng biến động ngược chiều với các biến giá.

CHƯƠNG 4. PHƯƠNG PHÁP THỰC NGHIỆM

4.1. Chuẩn hóa quy trình cho hai dataset

Để so sánh hai loại dữ liệu khác nhau, notebook cố tình dùng một pipeline tương tự: xác định feature và target, chia theo thời gian, chuẩn hóa, tạo sliding window, chuyển thành tensor/array sequence, huấn luyện RNN, dự đoán test và đánh giá bằng cùng metric.

Điều này làm giảm ảnh hưởng của khác biệt về cách chuẩn bị dữ liệu khi so sánh PyTorch với Keras.



Hình 13. Pipeline chuẩn bị chuỗi thời gian

Sơ đồ thể hiện hai nhánh Customer Behavior và AAPL cùng đi qua các bước Feature selection → Chronological split → StandardScaler → Sliding window → Sequence tensor.

4.2. Chia train – validation – test

Notebook dùng tỷ lệ 70% train, 15% validation và 15% test. Đây là chia theo vị trí thời gian, không phải random split. Với Customer Behavior có 305 mốc, train có 213, validation 46 và test 46. Sau sequence length 20, số mẫu hiệu dụng tương ứng là 193, 26 và 26. Với AAPL có 1.259 phiên, train có 881, validation 189 và test 189; sau sliding window còn 861, 169 và 169.

Dataset	Train	Validation	Test	Seq length	Sequence hiệu dụng
Customer Behavior	213	46	46	20	193 / 26 / 26

AAPL	881	189	189	20	861 / 169 / 169

4.3. Tách scaler cho feature và target

Notebook sử dụng feature_scaler và target_scaler riêng cho Customer Behavior; stock_feature_scaler và stock_target_scaler riêng cho AAPL. Cách làm này cho phép inverse_transform prediction về đơn vị ban đầu của target. Đây là lý do kết quả MAE/RMSE cuối cùng được thể hiện bằng đơn vị doanh thu hoặc giá, thay vì đơn vị chuẩn hóa.

4.4. Kiến trúc BasicRNN

BasicRNN trong PyTorch gồm nn.RNN(input_size, hidden_size, batch_first=True) và nn.Linear(hidden_size, 1). Trong forward, output và hidden được lấy từ RNN; output ở timestep cuối output[:, -1, :] được đưa qua Linear. Đây là mô hình many-to-one. Với cả hai dataset, input_size = 5 và hidden_size = 64.

Trích đoạn code BasicRNN.

```
class BasicRNN(nn.Module):

    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            batch_first=True
        )
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        output, hidden = self.rnn(x)
        last_output = output[:, -1, :]
        prediction = self.fc(last_output)
        return prediction
```

Giải thích: input_size bằng số feature của mỗi timestep. hidden_size bằng 64 là kích thước biểu diễn trạng thái. batch_first=True giúp tensor có dạng trực quan (batch, time, feature). Việc lấy output[:, -1, :] nghĩa là sử dụng biểu diễn của timestep cuối làm đại diện cho toàn bộ cửa sổ quá khứ. Linear biến vector 64 chiều thành một giá trị thực.

4.5. Huấn luyện

Cả hai mô hình PyTorch sử dụng MSELoss và Adam với learning rate 0,001. Số epoch là 50. DataLoader có batch size 16 và shuffle=False. Việc không shuffle giữ nguyên thứ tự các sequence trong quá trình huấn luyện, phù hợp với tinh thần dữ liệu tuần tự. Keras cũng dùng batch size 16, 50 epochs và shuffle=False.

4.6. Code tạo sequence

Trích đoạn code tạo sliding window.

```
def create_sequences(X, y, sequence_length):
    X_sequences = []
    y_targets = []

    for i in range(len(X) - sequence_length):
        X_sequences.append(
            X[i:i + sequence_length]
        )
        y_targets.append(
            y[i + sequence_length]
        )

    return np.array(X_sequences), np.array(y_targets)
```

Hàm này là điểm nối giữa phần xử lý dữ liệu và phần RNN. Nó không tạo thêm thông tin mới mà tái cấu trúc chuỗi thành các cặp input-target. Với sequence length 20, target luôn nằm ngay sau 20 timestep input.

4.7. Code đánh giá

Trích đoạn code đánh giá Customer Behavior bằng PyTorch.

```
mae = mean_absolute_error(actual_revenue, predictions_revenue)
```

```
mse = mean_squared_error(actual_revenue, predictions_revenue)
rmse = np.sqrt(mse)
r2 = r2_score(actual_revenue, predictions_revenue)
```

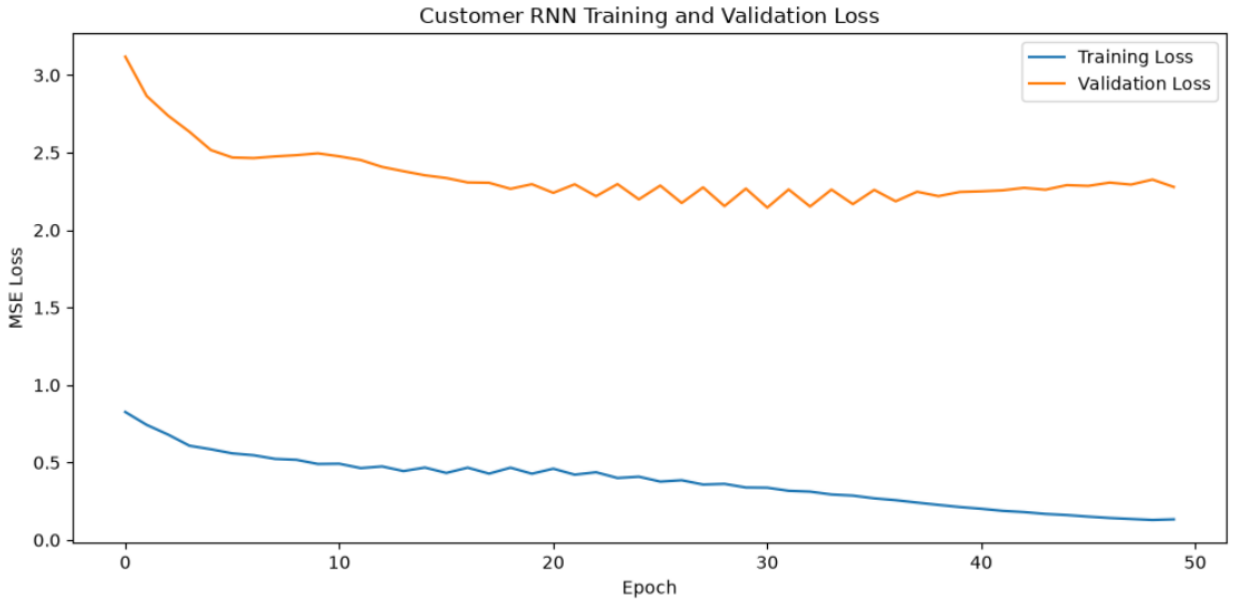
Code đánh giá đưa prediction và target về scale gốc trước khi tính metric. Đây là quyết định quan trọng vì MAE và RMSE khi tính trên dữ liệu đã chuẩn hóa sẽ không còn trực tiếp thể hiện sai số theo đơn vị doanh thu hoặc giá cổ phiếu.

CHƯƠNG 5. THỰC NGHIỆM PYTORCH TRÊN CUSTOMER BEHAVIOR

5.1. Chuẩn bị dữ liệu

Sau preprocessing, Customer Behavior có 305 mốc thời gian và 5 feature. Tỷ lệ chia 70/15/15 tạo ra 213/46/46 mẫu trước sliding window. Với cửa sổ 20, số sequence hiệu dụng là 193/26/26. Tập train chỉ có 193 sequence, vì vậy đây là một tập khá nhỏ đối với mạng nơ-ron. Đây là một yếu tố quan trọng khi diễn giải kết quả.

5.2. Huấn luyện và đường cong loss



Hình 14. Loss train/validation của PyTorch trên Customer Behavior

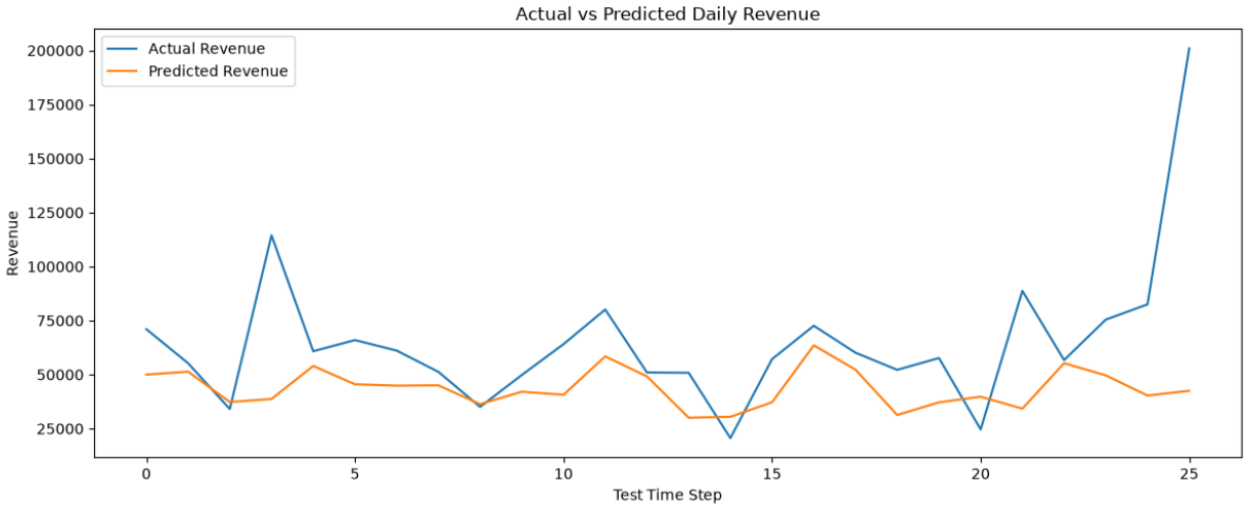
Nhận xét: Biểu đồ cho thấy Training Loss giảm đều từ khoảng 0,83 xuống gần 0,14 trong suốt quá trình huấn luyện. Trong khi đó, Validation Loss giảm mạnh ở những epoch đầu, sau đó dao động quanh mức 2,2–2,5 và có xu hướng tăng nhẹ về cuối. Khoảng cách lớn giữa Training Loss và Validation Loss cho thấy mô hình có dấu hiệu overfitting rõ rệt, tức là mô hình học tốt trên tập huấn luyện nhưng khả năng tổng quát hóa trên tập validation còn hạn chế.

Kết quả log cho thấy train loss giảm từ khoảng 0,8260 ở epoch 1 xuống mức thấp hơn nhiều ở cuối quá trình, trong khi validation loss vẫn dao động quanh hơn 2.2. Điều này cho thấy mô hình học rất tốt dữ liệu train nhưng khả năng khái quát hóa sang validation còn hạn chế. Một nguyên nhân trực tiếp là số sequence hiệu dụng chỉ có 193. Ngoài ra, doanh thu hằng ngày có biến động lớn và không chỉ phụ thuộc vào năm biến đang sử dụng.

5.3. Dự đoán và metric

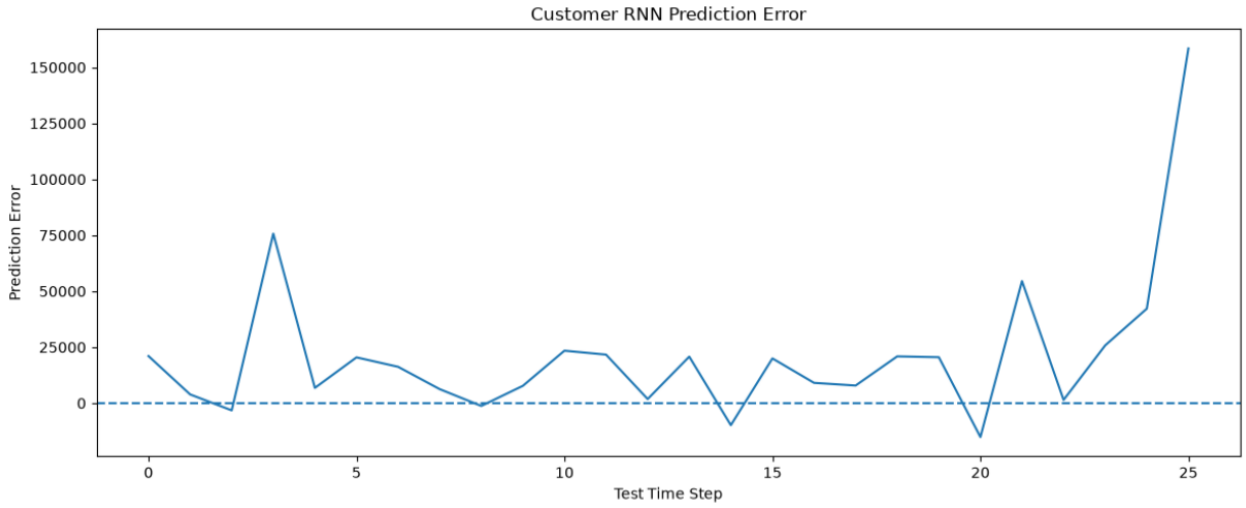
Metric	Giá trị
MAE	23.687,0410
MSE	1.566.219.043,4881
RMSE	39.575,4854
R^2	-0,4048

MAE khoảng 23,69 nghìn nghĩa là sai số tuyệt đối trung bình giữa doanh thu thật và doanh thu dự đoán trên test vào khoảng 23,7 nghìn đơn vị tiền tệ của dataset. RMSE lớn hơn MAE đáng kể, cho thấy một số sai số lớn có ảnh hưởng đáng kể; điều này phù hợp với MSE vì MSE bình phương sai số. R^2 = -0,4048 là dấu hiệu mô hình chưa giải thích tốt biến thiên test. Theo scikit-learn, R^2 có thể âm khi mô hình kém hơn mô hình hằng số dự đoán giá trị trung bình [7].



Hình 15. Doanh thu thật và dự đoán – PyTorch Customer Behavior

Nhận xét: Biểu đồ so sánh doanh thu thực tế và doanh thu dự đoán trên tập kiểm tra. Mô hình có thể nắm bắt được một phần xu hướng biến động của doanh thu, tuy nhiên giá trị dự đoán thường thấp hơn doanh thu thực tế, đặc biệt tại các thời điểm có doanh thu tăng đột biến. Điều này cho thấy mô hình còn hạn chế trong việc dự đoán các biến động lớn của doanh thu.



Hình 16. Sai số dự đoán – PyTorch Customer Behavior

Nhận xét: Sai số dự đoán biến động khá lớn giữa các thời điểm. Phần lớn sai số nằm ở mức thấp đến trung bình, nhưng xuất hiện một số thời điểm có sai số rất cao, đặc biệt ở cuối chuỗi. Các giá trị sai số dương lớn cho thấy mô hình có xu hướng dự đoán thấp hơn doanh thu thực tế tại những thời điểm doanh thu tăng mạnh.

5.4. Nhận xét

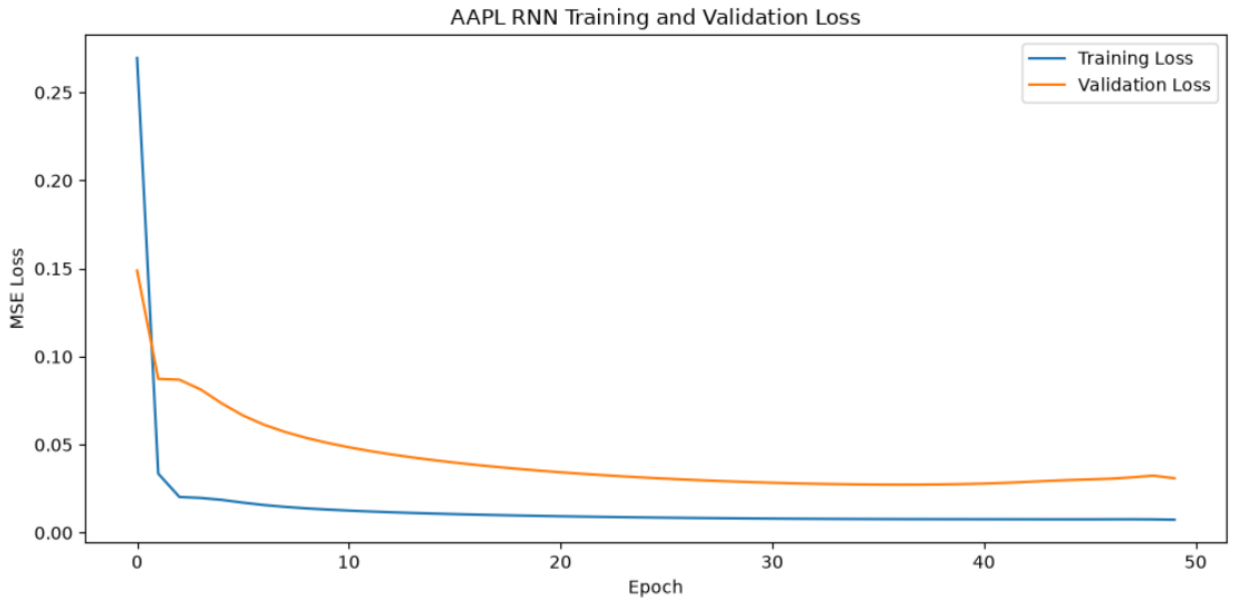
Kết quả PyTorch trên Customer Behavior hoàn thành đầy đủ pipeline dự báo nhưng chưa đạt chất lượng cao trên test. Đây không phải lỗi của quy trình tính toán. Ngược lại, nó cho thấy basic RNN với 20 ngày lịch sử và 5 feature chưa đủ để mô hình hóa toàn bộ biến động doanh thu. Đặc biệt, các yếu tố như khuyến mại, ngày lễ, danh mục sản phẩm, quốc gia, khách hàng mới/cũ và hiệu ứng mùa vụ chưa được đưa vào chuỗi cuối.

CHƯƠNG 6. THỰC NGHIỆM PYTORCH TRÊN AAPL

6.1. Chuẩn bị dữ liệu

AAPL có 1.259 phiên giao dịch, nhiều hơn đáng kể so với 305 mốc Customer Behavior. Sau chia 70/15/15, tập train có 881 phiên, validation 189 và test 189. Cửa sổ 20 tạo ra 861/169/169 sequence. Input gồm open, high, low, close và volume.

6.2. Huấn luyện



Hình 17. Loss train/validation của PyTorch trên AAPL

Nhận xét: Training Loss giảm rất nhanh từ khoảng 0,27 xuống gần 0,006 và tiếp tục ổn định ở mức thấp. Validation Loss cũng giảm đều trong phần lớn quá trình huấn luyện, từ khoảng 0,15 xuống gần 0,03, sau đó tăng nhẹ ở các epoch cuối.

Khoảng cách giữa hai đường Loss cho thấy mô hình có dấu hiệu overfitting nhẹ về cuối quá trình huấn luyện, nhưng nhìn chung mô hình đã hội tụ tốt.

So với Customer Behavior, AAPL có nhiều sequence hơn nên đường loss ổn định hơn. Validation loss giảm đáng kể từ epoch đầu xuống vùng khoảng 0,027 vào khoảng cuối giai đoạn giữa của training. Sau đó validation loss bắt đầu tăng nhẹ trong khi train loss vẫn giảm. Điều này cho thấy nếu dùng early stopping, có thể chọn checkpoint sớm hơn thay vì luôn lấy epoch 50.

6.3. Kết quả test

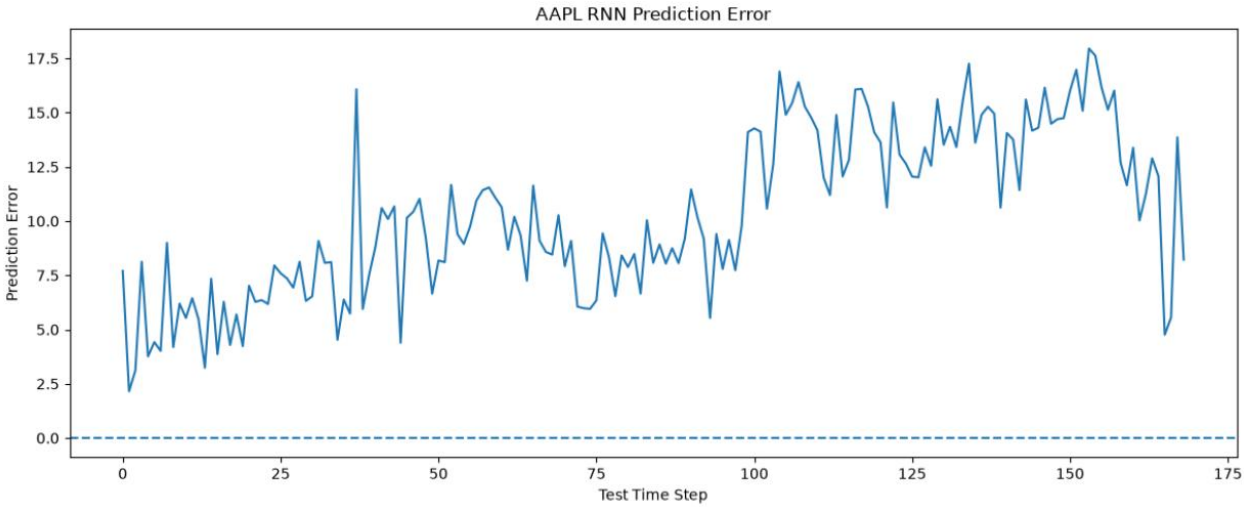
Metric	Giá trị
MAE	10,3069
MSE	120,2314
RMSE	10,9650
R^2	-0,1336

MAE khoảng 10,31 USD và RMSE khoảng 10,97 USD trong thí nghiệm cho thấy mô hình có sai số dự đoán đáng kể so với giá cổ phiếu. $R^2 = -0,1336$ vẫn âm, nhưng ít âm hơn Customer Behavior. Điều này cho thấy trong cấu hình hiện tại, basic RNN không giải thích tốt biến thiên của giá đóng cửa trong test.



Hình 18. Giá đóng cửa thật và dự đoán – PyTorch AAPL

Nhận xét: Biểu đồ cho thấy giá đóng cửa dự đoán có xu hướng biến động tương đối tương đồng với giá thực tế, đặc biệt trong việc nắm bắt xu hướng tăng giảm chung của chuỗi dữ liệu. Tuy nhiên, đường dự đoán thường thấp hơn giá thực tế và ít biến động hơn, đặc biệt tại các giai đoạn giá tăng mạnh hoặc giảm đột ngột. Điều này cho thấy mô hình RNN có khả năng học được xu hướng của giá cổ phiếu nhưng vẫn còn hạn chế trong việc dự đoán chính xác các biến động lớn.



Hình 19. Sai số dự đoán – PyTorch AAPL

Nhận xét: Sai số dự đoán có sự biến động mạnh giữa các thời điểm. Phần lớn sai số nằm ở mức thấp đến trung bình, nhưng xuất hiện một số thời điểm có sai số rất cao, đặc biệt ở gia đoạn 100-150.

6.4. Nhận xét

AAPL là bài toán khó hơn về mặt bản chất vì giá cổ phiếu không chỉ phụ thuộc vào năm biến nội tại của phiên gần nhất. Tin tức, thị trường chung, kết quả kinh doanh, lãi suất, tâm lý nhà đầu tư và nhiều yếu tố ngoại sinh không xuất hiện trong input. Vì vậy, việc R^2 vẫn âm là kết quả có thể xảy ra với một basic RNN có cửa sổ 20 phiên.

CHƯƠNG 7. THỰC NGHIỆM KERAS TRÊN CUSTOMER BEHAVIOR

7.1. Kiến trúc SimpleRNN

Keras sử dụng SimpleRNN với 64 units và activation tanh, sau đó Dense(1). Theo tài liệu Keras, SimpleRNN là lớp RNN fully-connected và nhận input ba chiều theo dạng batch, timesteps, feature [3]. Như vậy kiến trúc Keras tương ứng trực tiếp với BasicRNN trong PyTorch.

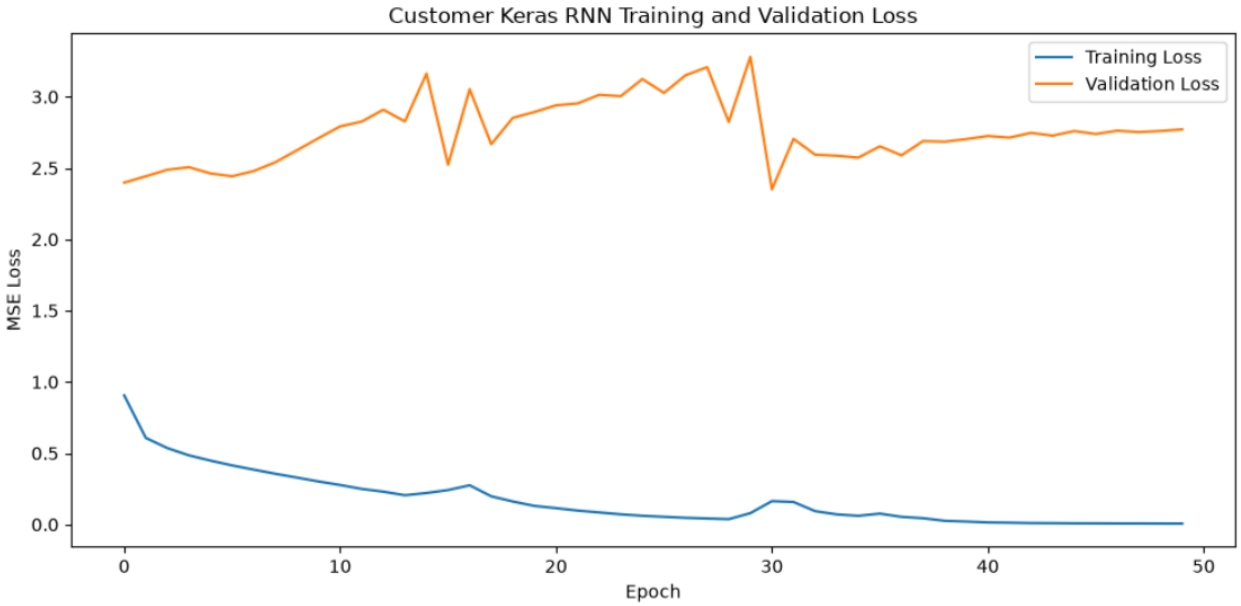
Trích đoạn code Keras Customer Behavior.

```
keras_customer_rnn = keras.Sequential([
    layers.Input(
        shape=(X_train_seq.shape[1], X_train_seq.shape[2])
    ),
    layers.SimpleRNN(64, activation="tanh"),
    layers.Dense(1)
])

keras_customer_rnn.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="mse"
)
```

Code sử dụng Input để cố định shape (20, 5), SimpleRNN(64) để xử lý chuỗi và Dense(1) để sinh dự đoán. Optimizer và loss được giữ giống PyTorch ở mức khái niệm: Adam và MSE.

7.2. Huấn luyện và đánh giá



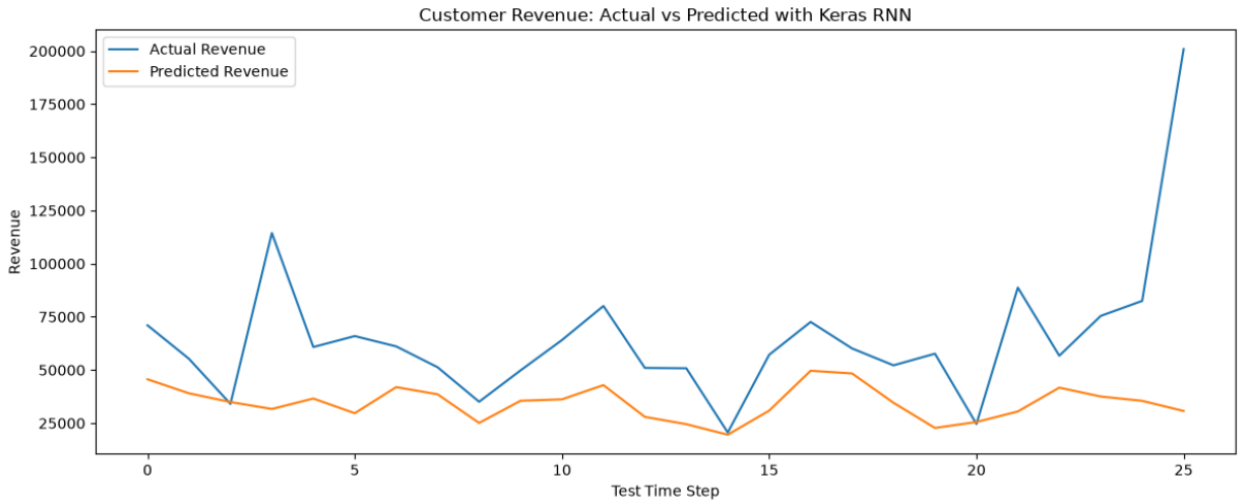
Hình 20. Loss train/validation của Keras trên Customer Behavior

Nhận xét: Training loss giảm nhưng validation loss không giảm tương ứng và dao động/tăng, cho thấy hạn chế khái quát hóa.

7.3. Kết quả

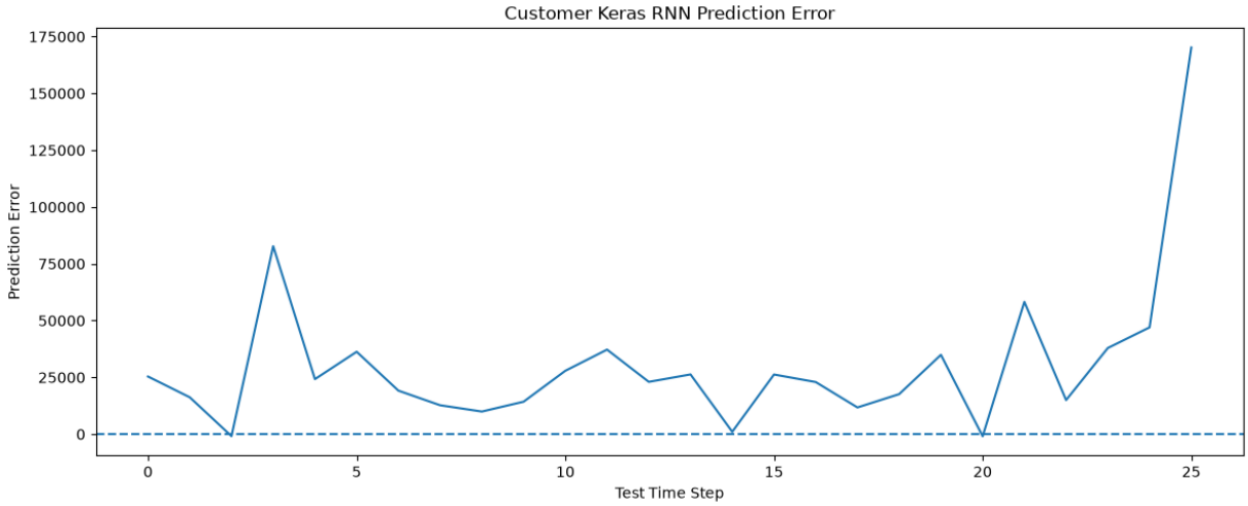
Metric	Giá trị
MAE	30.770,8299
MSE	2.037.844.288,3649
RMSE	45.142,4887
R^2	-0,8278

Keras đạt RMSE cao hơn PyTorch trên Customer Behavior. R^2 -0,8278 cũng thấp hơn -0,4048. Điều này cho thấy trong lần chạy thực nghiệm cụ thể này, Keras SimpleRNN có sai số test lớn hơn. Tuy nhiên, kết luận chỉ nên giới hạn trong cấu hình, seed và quá trình huấn luyện của notebook, không nên khái quát rằng Keras luôn kém PyTorch.



Hình 21. Doanh thu thật và dự đoán – Keras Customer Behavior

Nhận xét: Biểu đồ cho thấy doanh thu dự đoán có xu hướng biến động khác nhau với giá thực tế. Tuy nhiên, đường dự đoán thường thấp hơn giá thực tế và ít biến động hơn, đặc biệt tại các giai đoạn giá tăng mạnh hoặc giảm đột ngột.



Hình 22. Sai số dự đoán – Keras Customer Behavior

Nhận xét: Sai số dự đoán có sự biến động khá lớn ở một số giai đoạn. Hầu hết sai số thường ở mức thấp hoặc trung bình, nhưng lại tăng vọt lên cao ở giai đoạn cuối.

7.4. So sánh nhanh Customer Behavior

Khi giữ nguyên dữ liệu và sequence, PyTorch có MAE thấp hơn khoảng 7.084 đơn vị so với Keras và RMSE thấp hơn khoảng 5.567 đơn vị. MSE cũng thấp hơn. Sự khác

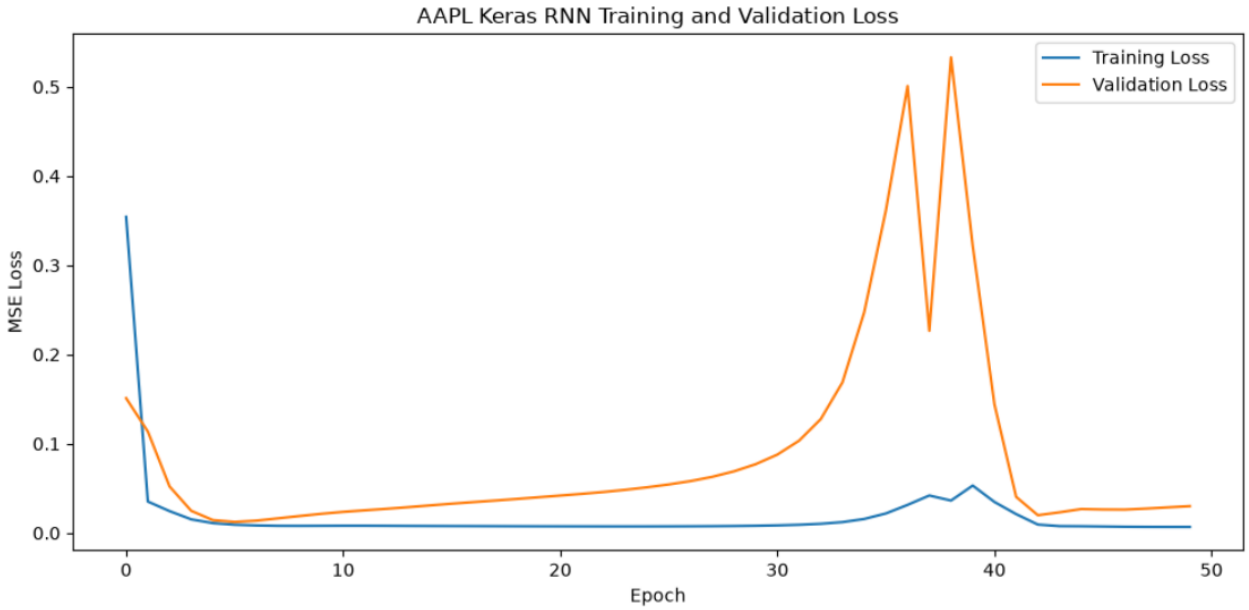
biệt có thể đến từ trạng thái khởi tạo, cách triển khai/khởi tạo trọng số, thứ tự tối ưu, đặc điểm numerical của hai framework và tính ngẫu nhiên. Vì notebook không cố định random seed chung cho cả hai framework, không nên diễn giải chênh lệch này như một bằng chứng tuyệt đối về chất lượng framework.

CHƯƠNG 8. THỰC NGHIỆM KERAS TRÊN AAPL

8.1. Cấu hình

Keras sử dụng SimpleRNN(64, tanh), Dense(1), Adam learning rate 0,001, batch size 16 và 50 epochs. Input giữ nguyên 20 timestep × 5 feature. Việc dùng cùng cấu hình với Customer Behavior giúp pipeline nhất quán nhưng không có nghĩa 50 epochs là tối ưu cho cả hai.

8.2. Huấn luyện



Hình 23. Loss train/validation của Keras trên AAPL

Nhận xét: Validation loss đạt mức thấp sớm, sau đó tăng dần trong khi train loss tiếp tục giảm; đây là dấu hiệu overfitting rõ.

Notebook cho thấy validation loss của Keras AAPL đạt khoảng 0,0124 ở epoch 6 rồi bắt đầu tăng. Trong khi đó training loss tiếp tục giảm. Nếu mục tiêu là tối ưu chất lượng

validation, checkpoint gần epoch 6 có thể hợp lý hơn epoch 50. Tuy nhiên, assignment hiện tại không yêu cầu early stopping, vì vậy notebook giữ nguyên 50 epochs để đảm bảo quy trình đơn giản và nhất quán.

8.3. Kết quả test

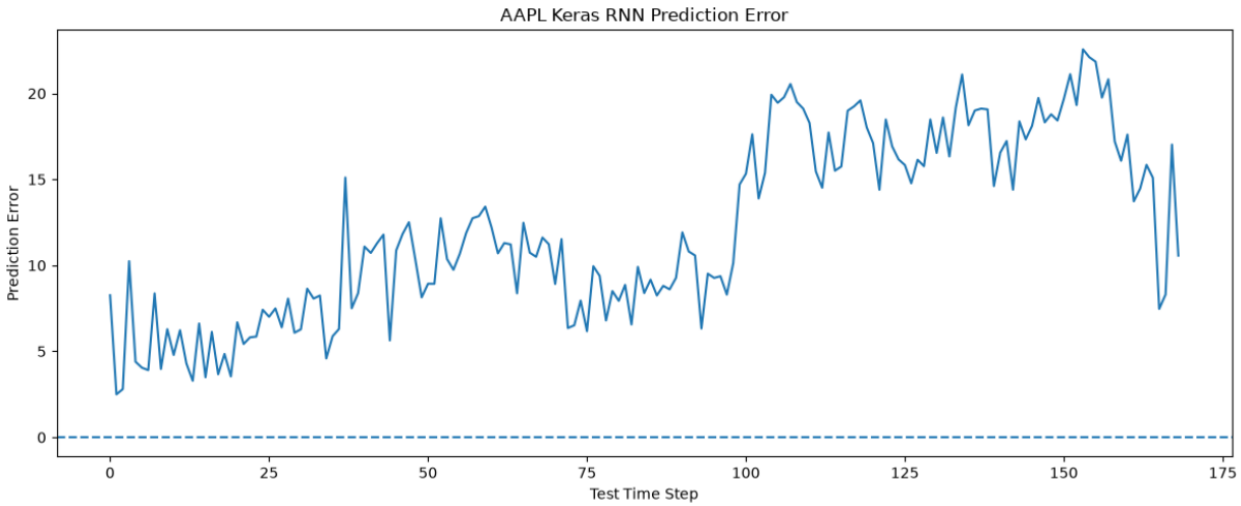
Metric	Giá trị
MAE	12,0690
MSE	172,7596
RMSE	13,1438
R^2	-0,6288

Keras AAPL có RMSE 13,1438, cao hơn PyTorch 10,9650. R^2 -0,6288 cũng thấp hơn đáng kể so với PyTorch -0,1336. Vì vậy, trong thực nghiệm này, PyTorch có sai số test nhỏ hơn đối với AAPL.



Hình 24. Giá AAPL thật và dự đoán – Keras

Nhận xét: Biểu đồ cho thấy giá AAPL thật và dự đoán có sự biến động tương đối tương đồng với nhau. Như các biểu đồ so sánh trước, thường giá dự đoán sẽ có xu hướng thấp hơn giá thật.



Hình 25. Sai số dự đoán AAPL – Keras

Nhận xét: Sai số dự đoán biến động đáng kể theo thời gian. Sai số ở giai đoạn đầu tương đối thấp nhưng tăng rõ rệt từ khoảng bước 100 trở đi, chủ yếu dao động trong khoảng 15–21. Điều này cho thấy độ chính xác dự đoán của mô hình giảm ở giai đoạn cuối của tập kiểm tra và mô hình chưa duy trì được độ ổn định trên toàn bộ chuỗi dữ liệu.

8.4. Nhận xét về warning TensorFlow

Khi kiểm tra TensorFlow, notebook ghi nhận TensorFlow 2.21.0 và cảnh báo GPU support không khả dụng trên native Windows đối với TensorFlow phiên bản mới. Warning này không làm bài toán thất bại; Keras vẫn huấn luyện và tạo kết quả. Trong phạm vi Assignment 06, tốc độ không phải mục tiêu chính nên việc chạy CPU vẫn đáp ứng yêu cầu.

CHƯƠNG 9. SO SÁNH PYTORCH VÀ KERAS

9.1. Bảng kết quả tổng hợp

Dataset	Framework	MAE	MSE	RMSE	R^2
Customer Behavior	PyTorch	23.687,0410	1.566.219.043,4881	39.575,4854	-0,4048

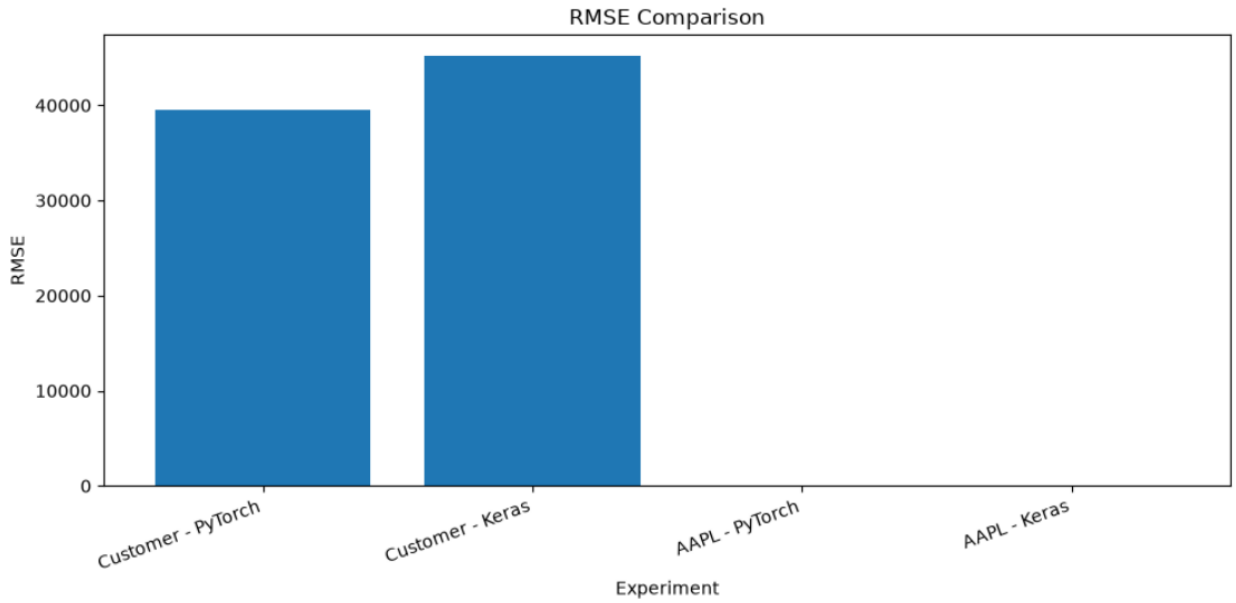
Customer Behavior	Keras	30.770,8299	2.037.844.288,3649	45.142,4887	-0,8278
AAPL Stock	PyTorch	10,3069	120,2314	10,9650	-0,1336
AAPL Stock	Keras	12,0690	172,7596	13,1438	-0,6288

9.2. So sánh theo Customer Behavior

Trong cùng dataset Customer Behavior, PyTorch có kết quả tốt hơn Keras theo cả bốn chỉ số được báo cáo. MAE giảm từ 30.770,83 xuống 23.687,04; RMSE giảm từ 45.142,49 xuống 39.575,49; MSE giảm từ 2,038 tỷ xuống 1,566 tỷ; R^2 tăng từ -0,8278 lên -0,4048. Theo tiêu chí RMSE mà notebook dùng để xác định framework có sai số thấp hơn, PyTorch được chọn trong lần chạy này.

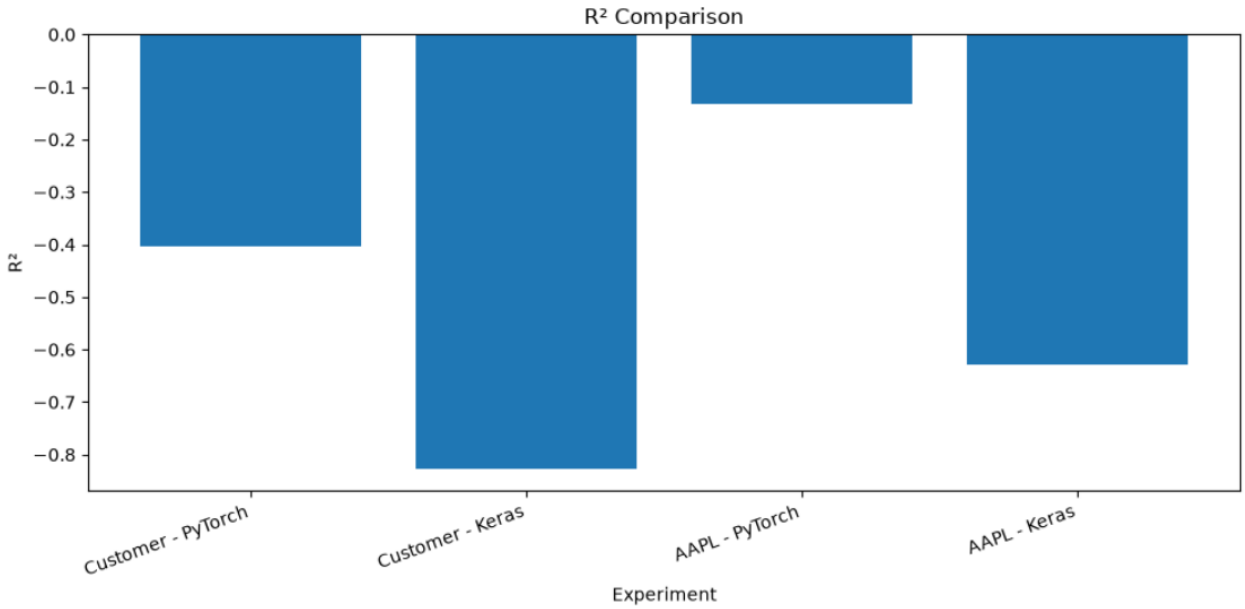
9.3. So sánh theo AAPL

AAPL cũng cho kết quả tương tự. PyTorch đạt MAE 10,3069 so với 12,0690 của Keras và RMSE 10,9650 so với 13,1438. R^2 của PyTorch là -0,1336, trong khi Keras là -0,6288. Vì vậy PyTorch có sai số test nhỏ hơn trong cả hai dataset.



Hình 26. So sánh RMSE của bốn thí nghiệm

Nhận xét: Biểu đồ cho thấy RMSE của mô hình Customer-Keras cao hơn Customer-PyTorch, cho thấy sai số dự đoán của Keras trên bài toán này lớn hơn. Các kết quả AAPL chưa thể hiện giá trị RMSE trên biểu đồ.



Hình 27. So sánh R^2 của bốn thí nghiệm

Nhận xét: Biểu đồ cho thấy sự chênh lệch của giá trị R^2 trong cùng 1 dataset ở PyTorch và Keras. Cả hai chỉ số của Customer ở cả PyTorch và Keras đều nhỏ hơn AAPL tương ứng.

9.4. Vì sao không so sánh RMSE giữa hai dataset?

RMSE có cùng đơn vị với biến mục tiêu nhưng hai target không cùng đơn vị và quy mô. Revenue của Customer Behavior có giá trị hàng chục nghìn đến hơn một trăm nghìn, trong khi close của AAPL nằm khoảng vài chục đến dưới hai trăm USD. Do đó RMSE 39.575 và RMSE 10,965 không thể dùng để kết luận dataset nào dễ hay khó chỉ bằng cách so sánh số tuyệt đối. Việc so sánh hợp lý là PyTorch với Keras trong cùng một dataset.

9.5. Ý nghĩa của R^2 âm

Cả bốn R^2 đều âm. Theo scikit-learn, R^2 có thể âm khi mô hình có hiệu năng kém hơn mô hình hằng số dự đoán trung bình [7]. Vì vậy, kết quả này cần được ghi nhận trung thực. Nó cho thấy cấu hình basic RNN hiện tại chưa tạo được khả năng giải thích biến thiên test tốt. Không nên diễn giải R^2 âm thành lỗi code.

9.6. Phân tích khả năng khái quát hóa

Các đường loss cho thấy một vấn đề chung: training loss thường giảm sâu hơn validation loss. Điều này cho thấy mô hình có khả năng học mẫu trong train nhưng khái quát hóa hạn chế. Customer Behavior đặc biệt nhạy với vấn đề này vì chỉ có 193 sequence train. AAPL có 861 sequence train nhưng vẫn xuất hiện overfitting ở Keras khi validation loss đạt cực tiểu sớm.

9.7. So sánh framework cần được diễn giải có điều kiện

Kết luận PyTorch tốt hơn Keras trong báo cáo phải được hiểu là kết luận cho bốn lần chạy trong notebook với cấu hình hiện tại. Không thể suy rộng thành phát biểu rằng PyTorch luôn tốt hơn Keras. Nếu cố định random seed, dùng nhiều lần chạy và áp dụng cùng chiến lược early stopping/hyperparameter tuning, thứ hạng có thể thay đổi. Trong assignment cơ bản, việc so sánh hiện tại đủ để minh họa cách triển khai cùng một ý tưởng RNN bằng hai framework.

9.8. Mã tạo bảng so sánh

Trích đoạn bảng tổng hợp.

```
comparison_results = pd.DataFrame([
    {
        "Dataset": "Customer Behavior",
        "Framework": "PyTorch",
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2
    },
    {
        "Dataset": "Customer Behavior",
        "Framework": "Keras",
        "MAE": customer_keras_mae,
        "MSE": customer_keras_mse,
        "RMSE": customer_keras_rmse,
        "R2": customer_keras_r2
    },
    {
        "Dataset": "AAPL Stock",
        "Framework": "PyTorch",
        "MAE": stock_mae,
        "MSE": stock_mse,
        "RMSE": stock_rmse,
        "R2": stock_r2
    },
    {
        "Dataset": "AAPL Stock",
        "Framework": "Keras",
        "MAE": stock_keras_mae,
        "MSE": stock_keras_mse,
        "RMSE": stock_keras_rmse,
        "R2": stock_keras_r2
    }
])
```

9.9. Trả lời trực tiếp bốn yêu cầu của Assignment 06

1. Yêu cầu 1 được đáp ứng bằng phần cơ sở lý thuyết về RNN, sequential data, timestep, hidden state, công thức toán học, input shape và mô hình BasicRNN chạy thử.
2. Yêu cầu 2 được đáp ứng bằng việc chọn Online Retail Business để biểu diễn Customer Behavior theo ngày và S&P 500 để chọn AAPL, kèm phân tích cấu trúc, thiếu dữ liệu, thống kê và trực quan hóa.
3. Yêu cầu 3 được đáp ứng bằng hai pipeline PyTorch hoàn chỉnh: Customer Behavior và AAPL, gồm preprocessing, sequence, training, prediction, metric và visualization.
4. Yêu cầu 4 được đáp ứng bằng hai pipeline Keras SimpleRNN tương ứng, sử dụng cùng sequence và metric để so sánh.

CHƯƠNG 10. ĐÁNH GIÁ, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

10.1. Điểm mạnh của thiết kế thực nghiệm

- Pipeline được xây dựng xuyên suốt từ dữ liệu thô đến kết quả dự đoán.
- Hai dataset đại diện cho hai loại chuỗi thời gian khác nhau.
- Chia dữ liệu theo thứ tự thời gian giúp tránh việc trộn tương lai vào quá khứ.
- Scaler được fit trên train trước khi transform validation/test.
- PyTorch và Keras dùng kiến trúc RNN cơ bản tương ứng.
- Có cả metric số và biểu đồ để đánh giá.
- Kết quả được giữ nguyên, kể cả khi R^2 âm.

10.2. Hạn chế của Customer Behavior

Hạn chế lớn nhất là số mốc thời gian sau aggregation chỉ còn 305. Khi dành 20 ngày cho mỗi cửa sổ, test chỉ còn 26 sequence. Một tập test 26 mẫu có thể khiến metric nhạy với một vài điểm bất thường. Ngoài ra, feature set cuối chỉ gồm năm biến tổng hợp. Các yếu tố như country, product category, seasonality, weekday, holiday và customer segment chưa được mã hóa.

Một điểm khác là CustomerID bị thiếu. Notebook giữ lại các dòng này vì vẫn có thể tính revenue và quantity, nhưng Customers chỉ đếm CustomerID không thiếu. Vì vậy feature Customers không phản ánh đầy đủ tổng số khách hàng nếu CustomerID thiếu mang tính hệ thống.

10.3. Hạn chế của AAPL

Dữ liệu AAPL chỉ có OHLCV của một cổ phiếu. Giá cổ phiếu chịu tác động của các biến ngoại sinh không có trong dataset. Ngoài ra, dự đoán trực tiếp close có thể khó hơn dự đoán return hoặc price change. Một cửa sổ 20 phiên có thể chưa đủ để biểu diễn xu hướng dài hạn.

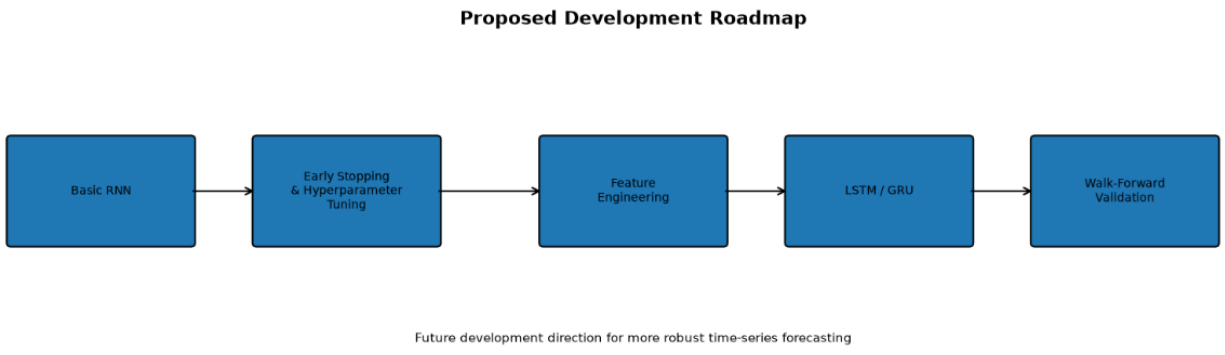
10.4. Hạn chế về reproducibility

Notebook không trình bày một quy trình cố định random seed cho cả NumPy, PyTorch và TensorFlow trước từng lần chạy. Vì vậy kết quả giữa hai framework có thể chịu ảnh hưởng của khởi tạo trọng số và tính ngẫu nhiên. Nếu mục tiêu là nghiên cứu so sánh framework ở mức nghiêm ngặt, cần chạy nhiều lần và báo cáo trung bình ± độ lệch chuẩn.

10.5. Hướng phát triển

- Dùng early stopping dựa trên validation loss.
- Thử sequence length 10, 20, 30 hoặc 60 và so sánh trên validation.
- Tối ưu hidden size và learning rate.
- Bổ sung đặc trưng lịch như weekday, month, holiday cho Customer Behavior.
- Bổ sung product/customer segmentation hoặc category-level features.
- Đối với AAPL, thử return, log-return, moving average, volatility và market index features.
- Sử dụng nhiều seed và nhiều lần chạy để có kết quả ổn định hơn.
- So sánh basic RNN với LSTM/GRU ở assignment nâng cao.
- Sử dụng walk-forward validation cho nghiên cứu forecasting nghiêm túc.
- Xem xét baseline đơn giản như last-value/naive forecast để đánh giá rõ giá trị tăng thêm của RNN.

10.6. Sơ đồ phát triển đề xuất cho phiên bản nâng cao



Hình 28. Hướng phát triển từ Basic RNN đến mô hình nâng cao

Nhận xét: Sơ đồ thể hiện Basic RNN → Early Stopping/Hyperparameter Tuning → Feature Engineering → LSTM/GRU → Walk-forward Validation. Đây là sơ đồ định hướng, không phải kết quả đã thực hiện trong Assignment 06.

CHƯƠNG 11. KẾT LUẬN

Assignment 06 đã được triển khai đầy đủ theo bốn yêu cầu chính. Phần lý thuyết giải thích RNN dưới góc nhìn dữ liệu tuần tự, timestep, hidden state, công thức cập nhật và dạng tensor. Phần thực nghiệm lựa chọn hai dataset phù hợp với yêu cầu: một dataset giao dịch để xây dựng Customer Behavior và một dataset chứng khoán để xây dựng AAPL time series.

Về quy trình dữ liệu, notebook cho thấy một bước quan trọng của time-series forecasting là chuyển dữ liệu thô thành sequence có cấu trúc. Online Retail ban đầu là dữ liệu transaction-level, nhưng sau cleaning và aggregation đã trở thành 305 mốc theo ngày. AAPL đã có cấu trúc theo phiên và được lọc từ 619.040 dòng toàn bộ dataset thành 1.259 phiên của một mã.

Về mô hình, PyTorch sử dụng BasicRNN gồm nn.RNN và Linear; Keras sử dụng SimpleRNN và Dense. Hai framework cùng xử lý sequence length 20, input size 5 và hidden size 64. Điều này tạo ra một thí nghiệm so sánh tương đối nhất quán.

Về kết quả, PyTorch đạt RMSE thấp hơn Keras trên cả Customer Behavior và AAPL trong lần chạy thực nghiệm này. Tuy nhiên, cả bốn R^2 đều âm, cho thấy basic RNN chưa đạt khả năng giải thích tốt trên test. Kết quả này cần được xem như một phát hiện về giới hạn của mô hình và thiết kế dữ liệu, không phải một lỗi cần che giấu.

Quan trọng nhất, bài tập đã hoàn thành mục tiêu học tập: từ hiểu RNN đến tự xây dựng pipeline time-series forecasting bằng hai framework phổ biến, biết chuẩn bị dữ liệu, tạo sequence, huấn luyện, đánh giá và phân tích kết quả. Đây là nền tảng trực tiếp để tiếp tục nghiên cứu các mô hình tuần tự nâng cao như LSTM và GRU.

TÀI LIỆU THAM KHẢO

[1] Elman, J. L. (1990). Finding Structure in Time. Cognitive Science, 14(2), 179–211. DOI: 10.1207/S15516709COG1402_1.

[2] PyTorch Documentation. torch.nn.RNN – Recurrent Neural Network. Tài liệu mô tả Elman RNN, hidden state và input shape.

[3] Keras Documentation. SimpleRNN layer – Recurrent layers. Tài liệu mô tả SimpleRNN, units, activation và input shape.

[4] scikit-learn Documentation. StandardScaler – Standardize features by removing the mean and scaling to unit variance.

[5] scikit-learn Documentation. mean_absolute_error – Mean absolute error regression loss.

[6] scikit-learn Documentation. mean_squared_error – Mean squared error regression loss.

[7] scikit-learn Documentation. r2_score – Coefficient of determination regression score function.

[8] Kaggle. Online Retail Business. Dataset page by Umerkk12.

[9] UCI Machine Learning Repository. Online Retail. Dataset ID 352.

[10] Kaggle. S&P 500 stock data. Dataset by Cam Nugent.

Liên kết nguồn

https://doi.org/10.1207/S15516709COG1402_1

https://docs.pytorch.org/docs/main/generated/torch.nn.RNN.html

https://keras.io/2/api/layers/recurrent_layers/simple_rnn/

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_error.html

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_squared_error.html

https://scikit-learn.org/1.6/modules/generated/sklearn.metrics.r2_score.html

https://www.kaggle.com/datasets/umerkk12/online-retail-business/data

https://archive.ics.uci.edu/dataset/352/online%2Bretail

https://www.kaggle.com/camnugent/sandp500/data

PHỤ LỤC A. SOURCE CODE VÀ GIẢI THÍCH CÁC KHỐI CODE

Phụ lục này giữ lại source code thực nghiệm quan trọng từ notebook rnn_a06.ipynb. Code được trích nguyên văn theo notebook ở các cell chính; các cell kiểm tra trung gian được giữ ở mức đại diện để báo cáo không biến thành một bản sao hoàn toàn của notebook. Notebook gốc là tài liệu thực thi chính.

Link source code đầy đủ: https://github.com/tnd17/Assignment-06-PTHTTM-N05

Cell 7 – Source code

```
# Import the libraries required for the basic RNN demonstration.

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn

from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

print("PyTorch version:", torch.__version__)
```

Giải thích: khối này khởi tạo các thư viện cần thiết cho phân tích dữ liệu, trực quan hóa, PyTorch, TensorFlow/Keras và metric hồi quy.

Cell 13 – Source code

```
# Define a simple PyTorch RNN model for sequence regression.

class BasicRNN(nn.Module):

    def __init__(self, input_size, hidden_size):
        super().__init__()

        self.rnn = nn.RNN(
            input_size=input_size,
```

```
            hidden_size=hidden_size,
            batch_first=True
        )

        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):

        output, hidden = self.rnn(x)

        last_output = output[:, -1, :]

        prediction = self.fc(last_output)

        return prediction


model = BasicRNN(
    input_size=5,
    hidden_size=32
)

print(model)
```

Giải thích: lớp BasicRNN là lõi của phần PyTorch. RNN xử lý sequence và Linear chuyển biểu diễn timestep cuối thành một giá trị dự đoán.

Cell 16 – Source code

```
# Load the customer transaction dataset using the appropriate file encoding.

CUSTOMER_DATA_PATH = "../data/OnlineRetail.csv"

customer_df = pd.read_csv(
    CUSTOMER_DATA_PATH,
    encoding="ISO-8859-1"
)

print("Dataset shape:", customer_df.shape)
customer_df.head()
```

Giải thích: đọc dữ liệu từ CSV. Dataset bán lẻ dùng encoding ISO-8859-1 để tránh lỗi ký tự; dataset cổ phiếu dùng encoding mặc định.

Cell 24 – Source code

```
# Convert the transaction date column to datetime using day-first format.

customer_df["InvoiceDate"] = pd.to_datetime(
    customer_df["InvoiceDate"],
    dayfirst=True
)

print(customer_df["InvoiceDate"].dtype)
```

Cell 26 – Source code

```
# Calculate the revenue generated by each transaction.

customer_df["Revenue"] = (
    customer_df["Quantity"] * customer_df["UnitPrice"]
)

customer_df[[
    "Quantity",
    "UnitPrice",
    "Revenue"
]].head()
```

Cell 31 – Source code

```
# Remove cancelled transactions from the sales dataset.

customer_sales = customer_df[
    ~customer_df["InvoiceNo"].astype(str).str.startswith("C")
].copy()

print("Original rows:", len(customer_df))
print("Rows after removing cancelled invoices:", len(customer_sales))
```

Giải thích: các cell này thực hiện cleaning và biến đổi dữ liệu thành dạng phù hợp với time series.

Cell 33 – Source code

```
# Remove transactions with non-positive unit prices.
```

```
customer_sales = customer_sales[
    customer_sales["UnitPrice"] > 0
].copy()

print("Rows after removing invalid prices:", len(customer_sales))
```

Giải thích: các cell này thực hiện cleaning và biến đổi dữ liệu thành dạng phù hợp với time series.

Cell 37 – Source code

```
# Aggregate transaction-level data into daily customer and business behavior.

daily_customer = (
    customer_sales
    .groupby("Date")
    .agg(
        Revenue=("Revenue", "sum"),
        Transactions=("InvoiceNo", "nunique"),
        Customers=("CustomerID", "nunique"),
        Quantity=("Quantity", "sum"),
        AvgUnitPrice=("UnitPrice", "mean")
    )
    .reset_index()
)

daily_customer["Date"] = pd.to_datetime(
    daily_customer["Date"]
)

daily_customer.head()
```

Giải thích: các cell này thực hiện cleaning và biến đổi dữ liệu thành dạng phù hợp với time series.

Cell 51 – Source code

```
# Split the daily time-series data chronologically into train, validation, and test sets.

n = len(daily_customer)

train_end = int(n * 0.70)
```

```
val_end = int(n * 0.85)

train_df = daily_customer.iloc[:train_end].copy()
val_df = daily_customer.iloc[train_end:val_end].copy()
test_df = daily_customer.iloc[val_end:].copy()

print("Train shape:", train_df.shape)
print("Validation shape:", val_df.shape)
print("Test shape:", test_df.shape)

print("\nTrain period:")
print(train_df["Date"].min(), "to", train_df["Date"].max())

print("\nValidation period:")
print(val_df["Date"].min(), "to", val_df["Date"].max())

print("\nTest period:")
print(test_df["Date"].min(), "to", test_df["Date"].max())
```

Giải thích: chia dữ liệu theo thứ tự thời gian để đảm bảo test đại diện cho tương lai so với train.

Cell 52 – Source code

```
# Fit the feature scaler on training data and transform validation and test data.

feature_scaler = StandardScaler()

X_train = feature_scaler.fit_transform(
    train_df[FEATURE_COLUMNS]
)

X_val = feature_scaler.transform(
    val_df[FEATURE_COLUMNS]
)

X_test = feature_scaler.transform(
    test_df[FEATURE_COLUMNS]
)

print("X_train shape:", X_train.shape)
print("X_val shape:", X_val.shape)
print("X_test shape:", X_test.shape)
```

Giải thích: fit scaler trên train rồi transform validation/test nhằm tránh data leakage.

Cell 53 – Source code

```
# Fit a separate scaler for the revenue target using only the training data.

target_scaler = StandardScaler()

y_train = target_scaler.fit_transform(
    train_df[[TARGET_COLUMN]]
).flatten()

y_val = target_scaler.transform(
    val_df[[TARGET_COLUMN]]
).flatten()

y_test = target_scaler.transform(
    test_df[[TARGET_COLUMN]]
).flatten()

print("y_train shape:", y_train.shape)
print("y_val shape:", y_val.shape)
print("y_test shape:", y_test.shape)
```

Giải thích: fit scaler trên train rồi transform validation/test nhằm tránh data leakage.

Cell 54 – Source code

```
# Create sliding-window sequences for next-day revenue prediction.

def create_sequences(X, y, sequence_length):
    X_sequences = []
    y_targets = []

    for i in range(len(X) - sequence_length):
        X_sequences.append(
            X[i:i + sequence_length]
        )

        y_targets.append(
            y[i + sequence_length]
        )

    return np.array(X_sequences), np.array(y_targets)
```

Giải thích: sliding window chuyển chuỗi thành các mẫu supervised có dạng 20 timestep × 5 feature.

Cell 60 – Source code

```
# Configure the PyTorch RNN model for customer behavior prediction.

INPUT_SIZE = X_train_seq.shape[2]
HIDDEN_SIZE = 64
OUTPUT_SIZE = 1

customer_rnn = BasicRNN(
    input_size=INPUT_SIZE,
    hidden_size=HIDDEN_SIZE
).to(device)

print(customer_rnn)
```

Giải thích: cấu hình mô hình RNN tương ứng. PyTorch dùng nn.RNN + Linear; Keras dùng SimpleRNN + Dense.

Cell 64 – Source code

```
# Train the customer RNN and record training and validation loss.

EPOCHS = 50

train_losses = []
val_losses = []

for epoch in range(EPOCHS):

    train_loss = train_one_epoch(
        customer_rnn,
        train_loader,
        criterion,
        optimizer,
        device
    )

    val_loss = evaluate_loss(
        customer_rnn,
        val_loader,
        criterion,
```

```
        device
    )

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    print(
        f"Epoch {epoch + 1:02d}/{EPOCHS} "
        f"- Train Loss: {train_loss:.4f} "
        f"- Val Loss: {val_loss:.4f}"
    )
```

Giải thích: huấn luyện mô hình và lưu training/validation loss để trực quan hóa và phát hiện overfitting.

Cell 66 – Source code

```
# Generate next-day revenue predictions on the customer test set.

customer_rnn.eval()

predictions_scaled = []

with torch.no_grad():

    for X_batch, _ in test_loader:

        X_batch = X_batch.to(device)

        predictions = customer_rnn(X_batch)

        predictions_scaled.extend(
            predictions.cpu().numpy().flatten()
        )

predictions_scaled = np.array(
    predictions_scaled
)

print("Number of predictions:", len(predictions_scaled))
```

Giải thích: chạy inference trên test set để tạo dự đoán.

Cell 68 – Source code

```
# Calculate regression metrics for customer revenue prediction.

mae = mean_absolute_error(
    actual_revenue,
    predictions_revenue
)

mse = mean_squared_error(
    actual_revenue,
    predictions_revenue
)

rmse = np.sqrt(mse)

r2 = r2_score(
    actual_revenue,
    predictions_revenue
)

print(f"MAE:  {mae:.4f}")
print(f"MSE:  {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"R²:   {r2:.4f}")
```

Giải thích: inverse transform về scale gốc rồi tính MAE, MSE, RMSE và R^2.

Cell 69 – Source code

```
# Visualize actual and predicted daily revenue on the customer test set.

plt.figure(figsize=(12, 5))

plt.plot(
    actual_revenue,
    label="Actual Revenue"
)

plt.plot(
    predictions_revenue,
    label="Predicted Revenue"
)

plt.title("Actual vs Predicted Daily Revenue")
```

```
plt.xlabel("Test Time Step")
plt.ylabel("Revenue")
plt.legend()

plt.tight_layout()
plt.show()
```

Cell 72 – Source code

```
# Load the S&P 500 stock market dataset.

STOCK_DATA_PATH = "../data/all_stocks_5yr.csv"

stock_df = pd.read_csv(
    STOCK_DATA_PATH
)

print("Dataset shape:", stock_df.shape)
stock_df.head()
```

Giải thích: đọc dữ liệu từ CSV. Dataset bán lẻ dùng encoding ISO-8859-1 để tránh lỗi ký tự; dataset cổ phiếu dùng encoding mặc định.

Cell 78 – Source code

```
# Select Apple Inc. (AAPL) stock data from the S&P 500 dataset.

aapl_df = stock_df[
    stock_df["Name"] == "AAPL"
].copy()

print("AAPL dataset shape:", aapl_df.shape)
aapl_df.head()
```

Cell 80 – Source code

```
# Convert the stock date column to datetime and sort the data chronologically.

aapl_df["date"] = pd.to_datetime(
    aapl_df["date"]
)
```

```
aapl_df = aapl_df.sort_values(
    "date"
).reset_index(drop=True)

print("Date range:")
print(aapl_df["date"].min())
print(aapl_df["date"].max())

aapl_df.head()
```

Giải thích: các cell này thực hiện cleaning và biến đổi dữ liệu thành dạng phù hợp với time series.

Cell 88 – Source code

```
# Select stock input features and the next-day closing price as the target.

STOCK_FEATURE_COLUMNS = [
    "open",
    "high",
    "low",
    "close",
    "volume"
]

STOCK_TARGET_COLUMN = "close"

STOCK_SEQ_LEN = 20

print("Input features:", STOCK_FEATURE_COLUMNS)
print("Target:", STOCK_TARGET_COLUMN)
print("Sequence length:", STOCK_SEQ_LEN)
```

Cell 89 – Source code

```
# Split the AAPL time series chronologically into training, validation, and test sets.

n_stock = len(aapl_df)

stock_train_end = int(n_stock * 0.70)
stock_val_end = int(n_stock * 0.85)

stock_train_df = aapl_df.iloc[
```

```
    :stock_train_end
].copy()

stock_val_df = aapl_df.iloc[
    stock_train_end:stock_val_end
].copy()

stock_test_df = aapl_df.iloc[
    stock_val_end:
].copy()

print("Train shape:", stock_train_df.shape)
print("Validation shape:", stock_val_df.shape)
print("Test shape:", stock_test_df.shape)

print("\nTrain period:")
print(
    stock_train_df["date"].min(),
    "to",
    stock_train_df["date"].max()
)

print("\nValidation period:")
print(
    stock_val_df["date"].min(),
    "to",
    stock_val_df["date"].max()
)

print("\nTest period:")
print(
    stock_test_df["date"].min(),
    "to",
    stock_test_df["date"].max()
)
```

Giải thích: chia dữ liệu theo thứ tự thời gian để đảm bảo test đại diện cho tương lai so với train.

Cell 90 – Source code

```
# Fit the stock feature scaler on training data and transform validation and test data.

stock_feature_scaler = StandardScaler()
```

```
stock_X_train = stock_feature_scaler.fit_transform(
    stock_train_df[STOCK_FEATURE_COLUMNS]
)

stock_X_val = stock_feature_scaler.transform(
    stock_val_df[STOCK_FEATURE_COLUMNS]
)

stock_X_test = stock_feature_scaler.transform(
    stock_test_df[STOCK_FEATURE_COLUMNS]
)

print("stock_X_train shape:", stock_X_train.shape)
print("stock_X_val shape:", stock_X_val.shape)
print("stock_X_test shape:", stock_X_test.shape)
```

Giải thích: fit scaler trên train rồi transform validation/test nhằm tránh data leakage.

Cell 91 – Source code

```
# Fit a separate scaler for the stock closing price target using only training data.

stock_target_scaler = StandardScaler()

stock_y_train = stock_target_scaler.fit_transform(
    stock_train_df[[STOCK_TARGET_COLUMN]]
).flatten()

stock_y_val = stock_target_scaler.transform(
    stock_val_df[[STOCK_TARGET_COLUMN]]
).flatten()

stock_y_test = stock_target_scaler.transform(
    stock_test_df[[STOCK_TARGET_COLUMN]]
).flatten()

print("stock_y_train shape:", stock_y_train.shape)
print("stock_y_val shape:", stock_y_val.shape)
print("stock_y_test shape:", stock_y_test.shape)
```

Giải thích: fit scaler trên train rồi transform validation/test nhằm tránh data leakage.

Cell 92 – Source code

```
# Create sliding-window sequences for next-day AAPL closing price prediction.

stock_X_train_seq, stock_y_train_seq = create_sequences(
    stock_X_train,
    stock_y_train,
    STOCK_SEQ_LEN
)

stock_X_val_seq, stock_y_val_seq = create_sequences(
    stock_X_val,
    stock_y_val,
    STOCK_SEQ_LEN
)

stock_X_test_seq, stock_y_test_seq = create_sequences(
    stock_X_test,
    stock_y_test,
    STOCK_SEQ_LEN
)

print("stock_X_train_seq:", stock_X_train_seq.shape)
print("stock_y_train_seq:", stock_y_train_seq.shape)

print("stock_X_val_seq:", stock_X_val_seq.shape)
print("stock_y_val_seq:", stock_y_val_seq.shape)

print("stock_X_test_seq:", stock_X_test_seq.shape)
print("stock_y_test_seq:", stock_y_test_seq.shape)
```

Giải thích: sliding window chuyển chuỗi thành các mẫu supervised có dạng 20 timestep × 5 feature.

Cell 97 – Source code

```
# Configure the PyTorch RNN model for AAPL stock prediction.

STOCK_INPUT_SIZE = stock_X_train_seq.shape[2]
STOCK_HIDDEN_SIZE = 64

stock_rnn = BasicRNN(
    input_size=STOCK_INPUT_SIZE,
    hidden_size=STOCK_HIDDEN_SIZE
).to(device)
```

```
print(stock_rnn)
```

Giải thích: cấu hình mô hình RNN tương ứng. PyTorch dùng nn.RNN + Linear; Keras dùng SimpleRNN + Dense.

Cell 99 – Source code

```
# Train the AAPL stock RNN and record training and validation loss.

STOCK_EPOCHS = 50

stock_train_losses = []
stock_val_losses = []

for epoch in range(STOCK_EPOCHS):

    train_loss = train_one_epoch(
        stock_rnn,
        stock_train_loader,
        stock_criterion,
        stock_optimizer,
        device
    )

    val_loss = evaluate_loss(
        stock_rnn,
        stock_val_loader,
        stock_criterion,
        device
    )

    stock_train_losses.append(train_loss)
    stock_val_losses.append(val_loss)

    print(
        f"Epoch {epoch + 1:02d}/{STOCK_EPOCHS} "
        f"- Train Loss: {train_loss:.4f} "
        f"- Val Loss: {val_loss:.4f}"
    )
```

Giải thích: huấn luyện mô hình và lưu training/validation loss để trực quan hóa và phát hiện overfitting.

Cell 101 – Source code

```
# Generate next-day closing price predictions on the AAPL test set.

stock_rnn.eval()

stock_predictions_scaled = []

with torch.no_grad():

    for X_batch, _ in stock_test_loader:

        X_batch = X_batch.to(device)

        predictions = stock_rnn(X_batch)

        stock_predictions_scaled.extend(
            predictions.cpu().numpy().flatten()
        )

stock_predictions_scaled = np.array(
    stock_predictions_scaled
)

print(
    "Number of predictions:",
    len(stock_predictions_scaled)
)
```

Giải thích: chạy inference trên test set để tạo dự đoán.

Cell 103 – Source code

```
# Calculate regression metrics for AAPL closing price prediction.

stock_mae = mean_absolute_error(
    stock_actual,
    stock_predictions
)

stock_mse = mean_squared_error(
    stock_actual,
    stock_predictions
)
```

```
stock_rmse = np.sqrt(
    stock_mse
)

stock_r2 = r2_score(
    stock_actual,
    stock_predictions
)

print(f"MAE:  {stock_mae:.4f}")
print(f"MSE:  {stock_mse:.4f}")
print(f"RMSE: {stock_rmse:.4f}")
print(f"R²:   {stock_r2:.4f}")
```

Giải thích: inverse transform về scale gốc rồi tính MAE, MSE, RMSE và R^2.

Cell 107 – Source code

```
# Import TensorFlow and Keras components for RNN modeling.

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print("TensorFlow version:", tf.__version__)
```

Giải thích: khối này khởi tạo các thư viện cần thiết cho phân tích dữ liệu, trực quan hóa, PyTorch, TensorFlow/Keras và metric hồi quy.

Cell 109 – Source code

```
# Build a Keras SimpleRNN model for customer behavior prediction.

keras_customer_rnn = keras.Sequential([
    layers.Input(
        shape=(
            X_train_seq.shape[1],
            X_train_seq.shape[2]
        )
    ),

    layers.SimpleRNN(
        64,
```

```
        activation="tanh"
    ),

    layers.Dense(1)
])

keras_customer_rnn.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="mse"
)

keras_customer_rnn.summary()
```

Giải thích: cấu hình mô hình RNN tương ứng. PyTorch dùng nn.RNN + Linear; Keras dùng SimpleRNN + Dense.

Cell 110 – Source code

```
# Train the Keras customer RNN and record training and validation loss.

CUSTOMER_KERAS_EPOCHS = 50
CUSTOMER_KERAS_BATCH_SIZE = 16

customer_history = keras_customer_rnn.fit(
    X_train_seq,
    y_train_seq,
    validation_data=(
        X_val_seq,
        y_val_seq
    ),
    epochs=CUSTOMER_KERAS_EPOCHS,
    batch_size=CUSTOMER_KERAS_BATCH_SIZE,
    shuffle=False,
    verbose=1
)
```

Giải thích: huấn luyện mô hình và lưu training/validation loss để trực quan hóa và phát hiện overfitting.

Cell 112 – Source code

```
# Generate next-day revenue predictions using the Keras customer RNN.

customer_predictions_scaled = (
    keras_customer_rnn.predict(
        X_test_seq,
        verbose=0
    ).flatten()
)

print(
    "Number of predictions:",
    len(customer_predictions_scaled)
)
```

Giải thích: chạy inference trên test set để tạo dự đoán.

Cell 113 – Source code

```
# Convert customer predictions to the original scale and calculate regression metrics.

customer_keras_predictions = target_scaler.inverse_transform(
    customer_predictions_scaled.reshape(-1, 1)
).flatten()

customer_keras_actual = target_scaler.inverse_transform(
    y_test_seq.reshape(-1, 1)
).flatten()

customer_keras_mae = mean_absolute_error(
    customer_keras_actual,
    customer_keras_predictions
)

customer_keras_mse = mean_squared_error(
    customer_keras_actual,
    customer_keras_predictions
)

customer_keras_rmse = np.sqrt(
    customer_keras_mse
)

customer_keras_r2 = r2_score(
```

```
    customer_keras_actual,
    customer_keras_predictions
)

print(f"MAE:  {customer_keras_mae:.4f}")
print(f"MSE:  {customer_keras_mse:.4f}")
print(f"RMSE: {customer_keras_rmse:.4f}")
print(f"R²:   {customer_keras_r2:.4f}")
```

Giải thích: inverse transform về scale gốc rồi tính MAE, MSE, RMSE và R^2.

Cell 117 – Source code

```
# Build a Keras SimpleRNN model for AAPL closing price prediction.

keras_stock_rnn = keras.Sequential([
    layers.Input(
        shape=(
            stock_X_train_seq.shape[1],
            stock_X_train_seq.shape[2]
        )
    ),

    layers.SimpleRNN(
        64,
        activation="tanh"
    ),

    layers.Dense(1)
])

keras_stock_rnn.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.001
    ),
    loss="mse"
)

keras_stock_rnn.summary()
```

Giải thích: cấu hình mô hình RNN tương ứng. PyTorch dùng nn.RNN + Linear; Keras dùng SimpleRNN + Dense.

Cell 118 – Source code

```
# Train the Keras stock RNN and record training and validation loss.

STOCK_KERAS_EPOCHS = 50
STOCK_KERAS_BATCH_SIZE = 16

stock_history = keras_stock_rnn.fit(
    stock_X_train_seq,
    stock_y_train_seq,
    validation_data=(
        stock_X_val_seq,
        stock_y_val_seq
    ),
    epochs=STOCK_KERAS_EPOCHS,
    batch_size=STOCK_KERAS_BATCH_SIZE,
    shuffle=False,
    verbose=1
)
```

Giải thích: huấn luyện mô hình và lưu training/validation loss để trực quan hóa và phát hiện overfitting.

Cell 120 – Source code

```
# Generate next-day closing price predictions using the Keras stock RNN.

stock_keras_predictions_scaled = (
    keras_stock_rnn.predict(
        stock_X_test_seq,
        verbose=0
    ).flatten()
)

print(
    "Number of predictions:",
    len(stock_keras_predictions_scaled)
)
```

Giải thích: chạy inference trên test set để tạo dự đoán.

Cell 121 – Source code

```
# Convert stock predictions to the original price scale and calculate regression metrics.

stock_keras_predictions = stock_target_scaler.inverse_transform(
    stock_keras_predictions_scaled.reshape(-1, 1)
).flatten()

stock_keras_actual = stock_target_scaler.inverse_transform(
    stock_y_test_seq.reshape(-1, 1)
).flatten()

stock_keras_mae = mean_absolute_error(
    stock_keras_actual,
    stock_keras_predictions
)

stock_keras_mse = mean_squared_error(
    stock_keras_actual,
    stock_keras_predictions
)

stock_keras_rmse = np.sqrt(
    stock_keras_mse
)

stock_keras_r2 = r2_score(
    stock_keras_actual,
    stock_keras_predictions
)

print(f"MAE:  {stock_keras_mae:.4f}")
print(f"MSE:  {stock_keras_mse:.4f}")
print(f"RMSE: {stock_keras_rmse:.4f}")
print(f"R²:   {stock_keras_r2:.4f}")
```

Giải thích: inverse transform về scale gốc rồi tính MAE, MSE, RMSE và R^2.

Cell 125 – Source code

```
# Create a comparison table for all PyTorch and Keras RNN experiments.

comparison_results = pd.DataFrame([
    {
        "Dataset": "Customer Behavior",
```

```
        "Framework": "PyTorch",
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2
    },
    {
        "Dataset": "Customer Behavior",
        "Framework": "Keras",
        "MAE": customer_keras_mae,
        "MSE": customer_keras_mse,
        "RMSE": customer_keras_rmse,
        "R2": customer_keras_r2
    },
    {
        "Dataset": "AAPL Stock",
        "Framework": "PyTorch",
        "MAE": stock_mae,
        "MSE": stock_mse,
        "RMSE": stock_rmse,
        "R2": stock_r2
    },
    {
        "Dataset": "AAPL Stock",
        "Framework": "Keras",
        "MAE": stock_keras_mae,
        "MSE": stock_keras_mse,
        "RMSE": stock_keras_rmse,
        "R2": stock_keras_r2
    }
])

comparison_results
```

Giải thích: gom toàn bộ metric của bốn thí nghiệm thành một DataFrame để so sánh thống nhất.

Cell 131 – Source code

```
# Identify the framework with the lower RMSE for each dataset.

customer_comparison = comparison_results[
    comparison_results["Dataset"] == "Customer Behavior"
]
```

```
stock_comparison = comparison_results[
    comparison_results["Dataset"] == "AAPL Stock"
]

customer_best_framework = customer_comparison.loc[
    customer_comparison["RMSE"].idxmin(),
    "Framework"
]

stock_best_framework = stock_comparison.loc[
    stock_comparison["RMSE"].idxmin(),
    "Framework"
]

print(
    "Lower-RMSE framework for Customer Behavior:",
    customer_best_framework
)

print(
    "Lower-RMSE framework for AAPL Stock:",
    stock_best_framework
)
```

Giải thích: xác định framework có RMSE thấp hơn trong từng dataset. Kết quả của notebook là PyTorch cho cả hai dataset.

PHỤ LỤC B. THÔNG TIN DATASET VÀ LIÊN KẾT

Dataset 1: Online Retail Business – Kaggle.

- File được sử dụng trong notebook: OnlineRetail.csv.
- Các trường gồm InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID và Country.
- Link nguồn: https://www.kaggle.com/datasets/umerkk12/online-retail-business/data

Dataset 2: S&P 500 stock data – Kaggle.

- File được sử dụng: all_stocks_5yr.csv.
- Các trường gồm date, open, high, low, close, volume và Name.
- Link nguồn: https://www.kaggle.com/camnugent/sandp500/data